Constraint-Grounded Witnesses for Explaining Graph Inference

Haolai Che, Mingjian Lu, Yangxin Fan, Hanchao Ma, Yinghui Wu
Case Western Reserve University
Cleveland, Ohio, USA
{hxc859, mxl1171, yxf451, hxm382, yxw1650}@case.edu

***Abstract*—Graph neural networks (GNNs) are widely used for prediction over graph data. Their subgraph explanations can preserve model predictions to support interpretable and responsible output. Nevertheless, such subgraphs may omit domain-specific patterns in incomplete graphs, hence remain misaligned with user-suggested evidence. This paper studies *constraint-grounded witness generation*, which grounds explanatory subgraphs (witnesses) to patterns expressed by user-specified graph constraints. We formulate *grounded witness* as a bi-level interpretation structure, which pairs a witness G_s, evaluated against the original model prediction, with an auxiliary grounded graph G_g. The latter combines observed supporting edges and explicitly labeled hypothesized edges. We propose quality measures for grounded witnesses in terms of conciseness, constraint alignment, and set-level coverage, and formulate a top-K witness generation problem. We establish NP-hardness and develop two algorithms: (1) ApxIChase uses bounded Chase-inspired search, and selectively reverse constraint matching to augment grounded witnesses with incremental verification. (2) HeuIChase uses tree matching and weighted arborescence expansion for fast witness construction with observed edges. Using real-world and synthetic benchmarks, we verify that our algorithms efficiently extract well-grounded interpretations and scale well with large graphs. We also demonstrate how the resulting grounded witnesses suggest contextual relationships for further investigation, in molecular property prediction and cybersecurity.**

I. Introduction

Machine learning models such as graph neural networks (GNNs) [1] have shown strong results in graph analysis. A graph model M can be viewed as a function that takes as input a representation of a graph $G = (X, A)$, where X (resp. A) is a matrix of node features (resp. adjacency matrix), and transforms G into a numerical matrix Z ("logits") through inference. The logits Z can then be post-processed into task-specific output for downstream analysis, such as labels for node classification. Given a test graph G and a graph model M, an "inference query" $Q(M, G)$ applies M over G to return the model output Z. Such a query may also return results $Q(M, V_T, G)$ for a set V_T of test nodes of interest in the graph.

While effective, trustworthy applications of graph models require more effective methods for interpreting their outputs. Model explanation has been studied to identify subgraphs as "witnesses" that help clarify why a result appears in the inference output, by ensuring that they preserve the same or a similar model output [2]. However, erroneous or incomplete graph data may yield witnesses that are faithful yet incomplete, inconsistent, or unconvincing. For example, molecular graph representations may contain ambiguous bond types, inconsistent charge annotations, or missing structural relations [3]. Consider the following motivating example.

Example 1: Figure 1(b) depicts a graph inference process in drug testing, over a molecular graph G that contains atoms (*e.g.,* v_1) and their bonds. For illustration, a 3-layered graph convolutional network (GCN) makes a node-level prediction at the highlighted atom v_t by aggregating representations of its 3-hop neighbors. To help a medical researcher interpret the output, an explainer computes a witness as a

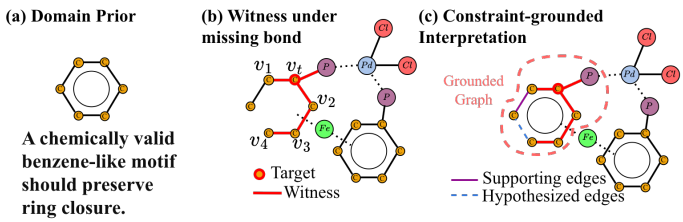


Fig. 1. Grounding a witness with a selected ring pattern. Purple edges provide observed context; blue dotted edges are grounding hypotheses. F_e denotes an iron atom outside the witness.

subgraph (highlighted by edges in red). It contains an open carbon chain around v_t (i.e., v_1–v_t–v_2–v_3–v_4) together with a nearby side branch, and it is sufficient for the model to return the same prediction on v_t as on the full graph. However, the observed graph lacks a ring-closing bond in the selected benzene-like pattern (Fig. 1(a)). The witness alone therefore does not show this pattern in full, which can make its structural context difficult to interpret. □

The above example separates two questions: *which observed edges preserve the prediction*, and *which domain patterns provide context for those edges*? A witness may answer the first question, yet lacks auxiliary links to suggest chemically meaningful molecular structure. Meanwhile, a graph constraint may supply a real molecular pattern for the chemical context of the witness, yet such context does not readily exist, due to possible incomplete graph data. The desired grounded interpretation should retain the witness, incorporate observed context, and also propose explicitly marked hypothetical links. Such a grounded witness indicates a domain compatible structure, justified by aligned constraints.

Example 2: Following Example 1, a data constraint, as illustrated in Fig. 1(a), specifies the co-occurrence of a five-edge carbon chain and its closing bond, forming a six-carbon motif ("benzene"). Fig. 1(c) illustrates how grounding augments the witness in Fig. 1(b). Specifically, the witness is extended with one observed supporting edge (purple) and one hypothesized ring-closing edge (blue dotted). The graph enclosed by the pink dashed boundary retains the witness around v_t. Its added edge makes the selected ring pattern structurally complete. The dotted bond indicates a proposed hypothetical bond aligned with the complete ring pattern specified by the constraint. The augmented witness can then be validated for investigation. □

These examples highlight the need for expressive witness structures that go beyond GNN explanations. We seek witness structures that should capture not only the data responsible for the model output, but also additional observed and hypothetical relationships, hence providing comprehensive context that is consistent with external graph data constraints. This motivates the following problem.

Constraint-grounded Interpretation for Graph Models. Given a graph G with a test node v_t, a GNN M, an inference query $Q(M, v_t, G)$ returning the model output over v_t on G, a constraint set Σ, and a bound K, the goal is to derive a top-K set of grounded witness pairs (G_s, G_g), where each G_g is a connected grounded graph containing a witness G_s, such that

- G_s captures a witness of G that clarifies the model output $Q(M, v_t, G)$, and can serve to assess interpretation quality to the model output (*e.g.,* faithfulness); and
- G_g "grounds" the witness G_s with a (hypothetical) context that supports at least one constraint in Σ, thereby aligning G_s with domain graph patterns specified by the constraint (*e.g.,* chemical compounds, social communities, or metabolic structures).

In this sense, our framework follows a *bi-level* view of grounded graph interpretation: G_s captures the model-relevant part of the interpretation, while G_g captures its grounding under data constraints. Hence, interpretation quality is evaluated on G_s, and grounding quality on G_g. Note that G_g need not be a subgraph of G; relations absent from G are grounding hypotheses suggested by the constraints.

Contributions. We make the following contributions.

(1) We formulate *constraint-grounded interpretation for GNN inference* as a bi-level problem, in which a witness graph captures the model-relevant part of the explanation and a grounded graph captures its grounding with declarative data constraints (Section III).

(2) We introduce quality measures for grounded witnesses in terms of constraint alignment, set-level coverage and conciseness. Based on the measures, we formulate *top-K grounded witness-generation problem*, which aims to compute K best grounded witnesses with size-bounded hypothetical relations, that also can be aligned to as many graph data constraints as possible (Section III).

(3) We present a principled computational framework for constrained-grounded witness generation that extends a Chase-inspired constraint enforcement to graphs (Section IV). We then develop two efficient implementations: ApxIChase, for general graphs, and HeuIChase for tree-like, sparse graphs, in Sections V and VI, respectively. We present optimized procedures to support witness verification, bounded constraint matching, and greedy top-K selection. We show an approximation guarantee of ApxIChase that is relative to the generated candidate pool, and present cost bounds for the arborescence method based on observed context grounding.

(4) In Section VII, we evaluate our methods on both real-world and synthetic graphs, and across multiple representative GNN architectures, showing that they produce high-quality, constraint-aligned witnesses with practical efficiency and substantially stronger grounded coverage than representative baselines.

Our contribution is the construction and selection of instance-level witness and context pairs with explicit representation of hypothesized relations, under supplied graph constraints. To our knowledge, this is the first approach to explicitly exploit graph constraints to enhance GNN explanation for expressive, grounded interpretations.

Related Work. We summarize related work below.

Graph Model Explanation. A large body of work explains GNN predictions by identifying influential subgraphs. Representative post-hoc methods include perturbation-based methods such as GNNExplainer [4], parameterized explainers such as PGExplainer [5], surrogate-based methods such as PGMExplainer [6], and causality-inspired approaches [7]–[9].

Recent work has also incorporated structured reasoning, but largely stops at translating subgraphs into rules rather than using rules as explicit grounding constraints [10]–[12]. Several approaches connect graph explanations with symbolic reasoning. GraphTrail derives logical rules from GNN predictions [11]; RAW-Explainer produces graph-based explanations for knowledge-graph predictions [12]. LogicXGNN constructs grounded logical rules from GNN behavior and evaluates their fidelity on graph instances [13].

These methods motivate the importance of graph-level evidence and explicit symbolic interpretation. Overall, they optimize behavioral fidelity, but do not ensure that the resulting explanations may indicate a context that conform to user-defined or domain-specific constraints. We address a complementary instance-level problem: supplied constraints guide the construction and selection of local witness and context pairs. Our bi-level representation records observed context and hypothesized edges separately, yielding interpretations that are both faithful to the model output and aligned with domain knowledge.

Provenance for Graph Queries. Provenance in data management studies how and why query results are derived from input data [14], and has primarily focused on structured queries. Recent work extends provenance to graph queries. [15] studies why-, why-not-, and why-rank questions for subgraph entity search over attributed graphs via query rewriting. [16] investigates why-questions for structural graph clustering, and [17] studies why-not questions in radius-bounded k-core search over geo-social networks.

These methods explain symbolic graph-query results, yet they do not directly produce witnesses and context that are grounded by graph data constraints. Our setting applies post-hoc witness verification to a fixed graph model and uses declarative patterns to construct contextual interpretations.

II. Inference, Constraints and Witness

A. Graph Model Inference

Graphs. A graph $G = (V, E)$ has a set of nodes V and a set of edges $E \subseteq V \times V$. Each node v has a type $v.l$, and carries a tuple $v.T$ of attributes and their values. The size of G, denoted as $|G|$, is the total number of its edges ($|G| = |E|$). Given a node v in G, the *L-hop neighbors* of v, denoted as $N^L(v)$, are the set of all nodes within L hops of v in G. The *L-hop subgraph* of a set of nodes $V_s \subseteq V$, denoted as $G^L(V_s)$, is the subgraph induced by the node set $\bigcup_{v_s \in V_s} N^L(v_s)$.

Graph Neural Networks. GNNs [1], [18] comprise a well-established family of deep learning models tailored for analyzing graph-structured data. GNNs generally employ a multi-layer message-passing scheme $\mathbf{H}^{(l+1)} = \sigma(\widetilde{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)})$, where $\mathbf{H}^{(l+1)}$ is the matrix of node representations at layer $l+1$, with $\mathbf{H}^{(0)} = \mathbf{X}$ being the input feature matrix. $\widetilde{\mathbf{A}}$ is a normalized adjacency matrix of an input graph G, which captures the topological feature of G. $\mathbf{W}^{(l)}$ is a learnable weight matrix at layer l (a.k.a "model weights"). σ is an activation function, such as ReLU.

The *inference process* of a GNN M with L layers takes as input a graph $G = (X, \widetilde{\mathbf{A}})$, and computes the embedding $\mathbf{H}_v^{(L)}$ for each node $v \in V$, by recursively applying the update function. The final layer's output $\mathbf{H}^{(L)}$ (a.k.a "output embeddings") is used to generate a task-specific *output*, by applying a post-processing layer (*e.g.,* a softmax function). We denote the task-specific output at node $v \in V$ by $M(v, G)$; for classification, this is the predicted class.

An inference query Q is a triple $Q = (M, v_t, G)$ where M is a fixed, deterministic GNN model, and v_t is a target node in a test graph G. The result of Q is the output $\mathbf{H}_{v_t}^{(L)}$ of M on v_t under

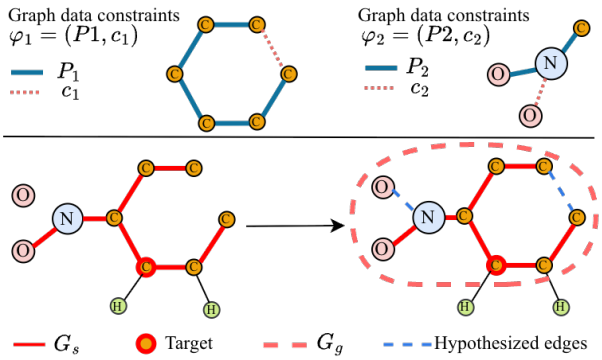


Fig. 2. Illustration of two data constraints(above) and a forward grounding step(below). A witness subgraph G_s (red) matching $\varphi_1 = (P_1, c_1)$ and $\varphi_2 = (P_2, c_2)$ is extended to a grounded graph G_g (pink dashed boundary) by enforcing the missing consequents as grounded edges (blue dashed). The resulting G_g contains co-occurring matches of both antecedents and consequents, and hence satisfies both constraints.

inference process of M over G,*i.e.,* $M(v_t, G)$. For fixed M and v_t, the output is determined by G and M as a function $G \to \mathbf{H}_{v_t}^{(L)}$.

A workload $\mathcal{Q} = (M, V_T, G)$ over node set $V_T \subseteq V$ contains a set of queries $Q(M, v_t, G)$, and returns $\mathbf{H}_{v_t}^{(L)}$ for each $v_t \in V_T$.

Example 3: Consider the node classification task in Fig. 1(b). Let M be a 3-layer GCN. An inference query $Q(M, v_t, G)$ requests the embedding $\mathbf{H}_{v_t}^{(3)}$ of atom node v_t in the compound graph G as the output of M. To answer Q, an inference process of M aggregates embeddings from the 3-hop neighborhood of v_t. □

B. Graph Data Constraints

We consider a class of graph data constraints (or simply, "constraints") in the form of a pair (P, c), where

(1) P ("antecedent") is a graph pattern that consists of a set of triple patterns. Each triple pattern is of the form $<u_s, u_p, u_o>$. Each subject node u_s has an entity type, and can be associated with a variable X_s and a conjunction of predicates of the form u_s op a, where a is a constant, and op is a comparison operator in $\{=, >, <, \geq, \leq, \neq\}$. The predicate u_p specifies the relation between u_s and u_o, while the object u_o denotes a literal value or another typed entity.

(2) c is a single triple pattern ("consequent") in the same form as in (1), and involves at least one node (variable) seen in P.

Semantics. A constraint $\varphi = (P, c)$ specifies a local co-occurrence pattern. A graph H satisfies φ, denoted by $H \models \varphi$, if there exists a graph homomorphism h that maps every triple in P and the consequent c to H under one variable assignment. Hence, $h(P)$ is a subgraph match of H (denoted as $h(P) \subseteq H$), and $h(c)$ is an edge match of c in H. Variables in c that are absent from P are assigned by the same homomorphism h.

We remark that φ poses *existential pattern satisfaction*. This meaning differs from universal graph dependencies, which enforces the co-occurrence of a matched consequence for all antecedent matches. For a constraint set Σ, $H \models \Sigma$ means $H \models \varphi$ for every $\varphi \in \Sigma$. In our context, H is an auxiliary graph G_g associated with a witness G_s; the satisfaction alone does not imply that a proposed edge is an observed fact already in G. In other words, G_s remains to be a subgraph of G, while G_g can contain an auxiliary edge not in G.

A constraint $\varphi = (P, c)$ is trivial when $c \in P$. For $\varphi = (P, c)$ and $\varphi' = (P', c)$ with the same consequent, $P \subseteq P'$ implies that any co-occurring match of $P' \cup \{c\}$ also contains a match of $P \cup \{c\}$. As will be discussed, we use this containment only to prune candidate matches for witness generation; the original set Σ remains the reference set for our quality measures such as coverage scores (see Section III). We consider nontrivial constraints.

Remarks. In this work, we focus on witness grounding under a given set of constraints; the problem of constraint discovery is orthogonal to our study. In practice, such constraints can be derived by graph association rules or dependency discovery [19], domain ontologies, such as domain-specific knowledge bases (e.g., UMLS for medicine) or RDF shapes (SHACL) [20]; or user-specified domain constraints, such as physics, chemistry, or finance.

Enforcement. Chase [21] is a powerful tool for query expressiveness, constraint reasoning and enforcement. Given a set of data constraints Σ and a dataset D, a Chase process is a general technique that transforms D such that $D \models \Sigma$. When Σ is used to express a query Q (hence a set of query constraints), Chase can be used for evaluating conjunctive queries. When Σ is defined as data quality constraints [22], Chase enforces data repairing process. Applying Chase twice yields Chase&BackChase [23], a query optimization technique where BackChase enforces inversed data constraints to rewrite queries. As will be discussed in Section IV, we exploit Chase and BackChase as algorithmic design principles, and introduce novel Chase-inspired constraint-grounded witness generation.

Example 4: Consider the witness subgraph G_s in Fig. 2, and two constraints $\varphi_1 = (P_1, c_1)$ and $\varphi_2 = (P_2, c_2)$. Here P_1 specifies a five-edge carbon chain, and c_1 requires the closing bond between its two endpoints, forming a six-carbon ring: $P_1 = \{\langle C_i, \textit{bond}, C_{i+1}\rangle \mid 1 \leq i \leq 5\}, c_1 = \langle C_6, \textit{bond}, C_1\rangle$. Moreover, P_2 requires that a nitrogen atom be bonded to an oxygen atom and a carbon atom, while c_2 adds another nitrogen–oxygen bond: $P_2 = \{\langle N, \textit{bond}, O_1\rangle, \langle N, \textit{bond}, C\rangle\}, c_2 = \langle N, \textit{bond}, O_2\rangle$. In Fig. 2, the red subgraph G_s matches antecedents P_1 and P_2. A grounding step then enforces the two consequents by introducing the missing relations as blue dashed edges, yielding the graph G_g on the right. As a result, G_g satisfies both constraints, *i.e.,* $G_g \models \{\varphi_1, \varphi_2\}$, under the local co-occurrence condition. These selected patterns specify the desired grounding, rather than universal chemical rules. □

C. Witnesses for Graph Inference

We characterize witnesses as explanatory subgraphs, following established factual and counterfactual explanation approaches.

Witness. Given an inference query $Q = (M, v_t, G)$ over a test node v_t and a graph ML model $M : G \to \mathbf{H}_v^{(L)}$, a subgraph G_s of G is a *witness* for the result of Q if G_s contains v_t, and:

- $Q(M, v_t, G) = Q(M, v_t, G_s)$ ("factual"); *or*
- $Q(M, v_t, G) \neq Q(M, v_t, G \setminus G_s)$ ("counterfactual")

Example 5: Given the inference query $Q(M, v_t, G)$ in Example 3, the red subgraph shown in Fig. 1(b) serves as a witness for the prediction on v_t. It captures the local carbon-chain fragment containing v_t, together with its linkage to a phosphorus atom, from which the 3-layer GCN M derives the representation $\mathbf{H}_{v_t}^{(3)}$ and the corresponding node-level prediction. In this sense, the witness isolates the local structure used by the model to produce the prediction. □

A factual witness is necessary to reconstruct the inference result of $Q(M, v_t, G)$. A counterfactual witness changes the latter if removed. A witness can be directly generated by a post-hoc explainer that ensures factual [4] or counterfactual [24] explanations, or induced from a learned masked adjacency matrix [5].

Remarks. We distinguish $Q(M, v_t, G)$, the output-layer embedding $\mathbf{H}_{v_t}^{(L)}$ from a predicted class probability vector. The latter can be readily obtained by applying a classification layer to $\mathbf{H}_{v_t}^{(L)}$. We follow established GNN explanation benchmarks and use the predicted class for task-specific fidelity to evaluate witnesses (see Section VII).

III. Grounding Witnesses with Constraints

Witnesses alone may not align with meaningful or user suggested domain context, as they are only faithful to model behavior [4]. We next describe how data constraints can be exploited to ground witnesses. To this end, we introduce a notion of k-grounded witness.

k-Grounded Witness. Let $G_s \subseteq G$ be a verified witness for $Q(M, v_t, G)$. A *grounded graph* G_g of G_s is a connected auxiliary graph that contains the following:

- a witness G_s, *i.e.,* $G_s \subseteq G_g$;
- a set of observed context edges in G ("supporting edges"), and
- a set of hypothesized edges $\Delta E = E(G_g) \setminus E$, *i.e.,* the edges in G_g that are not seen in G ($E(G_g)$ means the edge set of G_g).

Given a data constraint $\varphi \in \Sigma$, and an integer $k \geq 0$, the witness G_s is *k-grounded* by φ (or φ *supports* G_s), written as $G_s \models_k \varphi$, if there exists a grounded graph G_g of G_s, such that $G_g \models \varphi$ and $|\Delta E| \leq k$. A k-grounded witness indicates a hypothesized context G_g, which suggests at most k new relations ΔE to have the context supported by the user-defined pattern P of φ. The factual validity of proposed ΔE can then be separately investigated by the user.

A witness is k-grounded by a subset of constraints $\widehat{\Sigma} \subseteq \Sigma$, if for each $\varphi \in \widehat{\Sigma}$, a corresponding G_g satisfies the above condition. Each constraint may support a different grounded graph and edge set. For a single G_g supported by several constraints $\widehat{\Sigma}$, its hypothesized edges ΔE has a size bounded by the same budget k applied to all constraints in $\widehat{\Sigma}$. The observed input graph G remains unchanged, and no hypothesized edges will be inserted to G.

We now introduce a bi-level interpretation structure. Given a set of constraints Σ, integer k, and inference query $Q(M, v_t, G)$, a feasible *grounded witness pair* is a pair (G_s, G_g) that contains a k-grounded witness G_s and a grounded graph G_g of G_s.

Example 6: Consider $\Sigma = \{\varphi_1, \varphi_2\}$ in Fig. 2, where φ_1 enforces the carbon ring closure and φ_2 regulates the nitrogen–oxygen bonding pattern. Let G_s be the witness (red bold edges) that matches P_1 and P_2, and let $G_g = G_s \oplus \Delta E$ be the extended graph enclosed by the pink dotted boundary. Here ΔE denotes the blue dashed links that do not appear in the original G, which correspond to the consequents c_1 and c_2, respectively. Specifically, the added edge (C_6, C_1) enables $G_g \models \varphi_1$ by closing the carbon ring, and (N, O_2) ensures $G_g \models \varphi_2$ by completing the nitrogen–oxygen linkage. Hence $G_s \models_1 \varphi_1$ and $G_s \models_1 \varphi_2$, indicating that G_s is 1-grounded by Σ. □

Bi-level Quality Measures. As many witnesses can be grounded under constraints by multiple grounded graphs, we quantify the quality of a bi-level pair (G_s, G_g), consistently from two complementary levels: the quality of an individual witness pair, and the collective quality of a set of pairs in terms of supported constraints.

Quality of a witness pair. We introduce a pair-level quality measure for a pair (G_s, G_g), in terms of conciseness and alignment:

$$q(G_s, G_g) = \alpha \cdot \mathsf{conc}(G_s) + \beta \cdot \mathsf{aln}(G_s, G_g),$$

where $\alpha, \beta \geq 0$ are tunable weights. We elaborate the conciseness $\mathsf{conc}(G_s)$, and the alignment score $\mathsf{aln}(G_g)$ respectively.

Conciseness Let $G^L(v_t)$ be the L-hop input neighborhood of target v_t. We measure witness conciseness by

$$\mathsf{conc}(G_s) = 1 - \frac{|G_s|}{|G^L(v_t)|},$$

where $|G_s|$ denotes the number of edges of a G_s; similarly for $|G^L(v_t)|$. This measure favors compact witnesses among those satisfying the factual or counterfactual condition.

Alignment. For a feasible pair (G_s, G_g), let $\widehat{\Sigma} \subseteq \Sigma$ be a non-empty set of constraints that support G_s ($|\widehat{\Sigma}| > 0$), all under the same budget $k > 0$. While the hypothesized edge set ΔE has bounded size k, one would prefer grounded context that introduces smaller ΔE, to reduce downstream human attention and validation effort. We therefore introduce an alignment score that penalizes a normalized cost on accumulated hypothesized edge size over $\widehat{\Sigma}$:

$$\mathsf{aln}(G_s, G_g) = 1 - \frac{|\Delta E|}{k \cdot |\widehat{\Sigma}|},$$

This measure favors pairs with higher alignment score (hence lower alignment cost). For k=0, a grounded witness requires no hypothetical edges, hence has alignment score 1. If there is no supporting constraint ($|\widehat{\Sigma}| = 0$), the alignment score is trivially 0.

Coverage of Witness pair sets. A set of witness pairs grounded by more constraints often suggest a more comprehensive context, supported by multiple domain patterns in their antecedents. We thus introduce a set-level measure. For a set of witness pairs $\mathcal{P} = \{(G_s^1, G_g^1), \ldots, (G_s^K, G_g^K)\}$, we define its covered constraint set as

$$\mathsf{cov}(\mathcal{P}) = \frac{|\mathsf{CovSet}(\mathcal{P})|}{|\Sigma|}.$$

where $\mathsf{CovSet}(\mathcal{P}) \subseteq \Sigma$ is the set of all the constraints that support some witness pair in $\mathcal{P}$; the higher, the better.

Overall Quality Objective. We favor a set of grounded witness pairs whose individual members are concise and well aligned, while collectively covering as many data constraints as possible. Therefore, we define the overall quality score of a witness pair set $\mathcal{P}$ as

$$F(\mathcal{P}) = \sum_{(G_s, G_g) \in \mathcal{P}} q(G_s, G_g) + \gamma \cdot \mathsf{cov}(\mathcal{P}),$$

where $\gamma \geq 0$ is a tunable weight on the set-level coverage term.

An Axiomatic Analysis. We justify the set objective $F(\mathcal{P})$ by the following properties. For any inference query $Q(M, v_t, G)$, let $\mathcal{P}^*$ be an optimal witness pair set that maximizes $F(\mathcal{P})$ under the cardinality constraint $|\mathcal{P}| \leq K$. The function $F(\mathcal{P})$ has the following properties.

Scale invariance. $\mathcal{P}^*$ remains optimal if the component scores in $q(G_s, G_g)$ and the coverage term $\mathsf{cov}(\mathcal{P})$ are scaled by a common constant, provided that the corresponding weights are scaled accordingly. This ensures that the optimal witness pair set is invariant to the normalization of user preference weights.

Non-monotonicity. $F(\mathcal{P})$ is not necessarily monotone with respect to the sizes of individual witnesses or grounded graphs. Indeed, selecting larger witnesses does not necessarily improve the overall quality, since additional edges may reduce conciseness without proportionally improving alignment or coverage.

Locality. The objective $F(\mathcal{P})$ respects a locality principle. Specifically, the local quality of each selected pair $(G_s, G_g) \in \mathcal{P}$ is captured by $q(G_s, G_g)$, while interactions among selected pairs arise only through the set-level coverage term $\mathsf{cov}(\mathcal{P})$. No information outside the selected witness set and their corresponding grounded graphs

affects the overall quality evaluation here. This reflects a pragmatic semi-closed-world view: we evaluate grounded interpretations with respect to the observed graph and a bounded set of hypothesized edges, rather than reasoning over arbitrary unseen facts or assuming that the observed graph is already complete.

Problem Statement. Given $Q(M, v_t, G)$, a finite constraint set Σ, an edge insertion budget k, and an integer $K \geq 1$, the problem of *Top-K grounded witness generation* computes a set of at most K grounded witness pairs $\mathcal{P} = \{(G_s^1, G_g^1), \ldots, (G_s^K, G_g^K)\}$, such that (1) each pair $(G_s, G_g) \in \mathcal{P}$ consists of a k-grounded witness G_s and a corresponding grounded graph G_g, supported by at least one constraint in Σ; and (2) $\mathcal{P}$ maximizes the overall objective as:

$$\mathcal{P}^* = \arg \max_{\substack{\mathcal{P}' \subseteq \mathcal{C} \\ |\mathcal{P}'| \leq K}} F(\mathcal{P}'),$$

where $\mathcal{C}$ denotes the set of feasible candidate grounded witness pairs. This problem is non-trivial as verified by the hardness result below (see detailed hardness analysis in [25]).

Hardness Analysis. We consider the decision version of top-K witness generation: given an inference query $Q(M, v_t, G)$, a budget k, a set of constraints Σ, and an integer K, determine whether there exists a grounded witness set $\mathcal{P}$ with $0 < |\mathcal{P}| \leq K$.

Lemma 1: *Deciding whether a feasible grounded witness pair exists is NP-hard in combined graph and pattern sizes, even for a fixed GNN M and constants $K = 1$, $k = 0$ and $|\Sigma| = 1$.* □

Proof sketch: We show the result by constructing a reduction from *Clique*. We encode an instance of the r-clique problem with a co-occurrence pattern whose size depends on r. A fixed GNN produces a constant prediction, hence a singleton target is a verified witness. Setting $k = 0$, there exists a grounded context captured by a grounded graph if and only if the observed graph G contains the required clique. We show the full proof in [25]. □

A simple solution. We start with a simple algorithm, denoted as naiveChase, that combines fixed-point Chase enforcement, graph pattern matching, and witness verification to generate k-grounded witnesses. The procedure follows an "enforce-and-ground" process.

(1) Given a query $Q(M, v_t, G)$ and a constraint set Σ, naiveChase first enforces Σ on the local input graph $G^L(v_t)$ via a fixed-point computation. This is done by a variant of Chase on graphs, denoted as I-Chase, which iteratively inserts an edge that matches a consequent c whenever a subgraph match of the antecedent P is found in G, and terminates when no constraint can be enforced. As I-Chase runs over a fixed set of edges in G, each enforcement step inserts a missing edge between existing nodes in G, yielding a result graph G'.

The following simple yet important lemma establishes termination guarantee for I-Chase with graph constraints.

Lemma 2: *Given a finite graph G and a finite set of constraints Σ,* I-Chase *terminates after at most $|R||V|^2$ - $|E|$ distinct labeled directed edge insertions, where R is the finite set of edge labels.* □

(2) For each $\varphi = (P, c)$ and each subgraph G_g that matches P, naiveChase invokes a verification procedure VerifyWitness (details in appendix [25]) to check if a subgraph of G_g has a witness G_s for $Q(M, v_t, G)$. For each verified witness G_s, it then derives candidate grounded edges from the chased graph G'. Since $G' \models \Sigma$ as guaranteed by the correctness of Chase process, any verified witness G_s that contains a triggered match of P admits a corresponding grounded graph G_g for φ in G'.

While naiveChase can derive a set of grounded witnesses (see Analysis in [25]), it remains infeasible for large real-world G with missing edges. (1) Fixed-point Chase enforcement of Σ is inherently expensive, due to subgraph search to match P, and witness extraction from subgraphs of G_g. (2) The resulting pairs (G_s, G_g) from G' may contain excessively many hypothetical edges not in G, hence may suggest less meaningful or irrelevant context to inspect.

We next introduce efficient algorithms to generate grounded witness pairs, without an exhaustive enforcement of Σ over input graph.

IV. Witness Generation: Overview

We introduce an algorithmic framework, denoted as Inf-Chase. Our framework uses a two phases, Chase& BackChase inspired strategy, which first localize promising witnesses in G, and then inversely enforce constraints to ground them with domain patterns in their antecedents. Specifically, (1) We explore observed witness W_s as candidates that may contribute to a context supported by a constraint $\varphi = (P, c)$, by checking if W_s contains edges that match c. (2) We then augment W_s with compatible antecedent matches by enforcing "inversed" constraints $\varphi^{-1} = (c, P)$. This step induces hidden hypothetical edges from nearby matches of P. We retain such edges in an auxiliary interpretation, keeping the input graph G unchanged during the process.

Outline. Our algorithm is a two phase below:

(1) A **Chase** phase, which enforces constraints from Σ over the L-hop subgraph $G^L(v_t)$ of v_t whenever they are applicable, via a procedure **L-Chase**, a variant of I-Chase that performs bounded rounds of reasoning (see "L-Chase"); and

(2) A **Backchase** phase, which uses an ***inverse*** set of constraints derived from Σ to ground each witness in $\mathcal{G}_s$ under budget k. This is supported by a procedure **L-Backchase**, which uses an input inverse constraint set $\Sigma_b \subseteq \Sigma^{-1}$ on G_s, together with an edge update budget k, to obtain a graph G_g that grounds the witness G_s. Here each data constraint $\varphi^{-1} = (c, P)$ in Σ_b specifies a reverse search direction for some $\varphi = (P, c)$ in Σ: starting from a match of c, it searches for a matching P within budget k. It does not assert the logical converse of the original rule. Putting together, the process derives a K-set of grounded witness pairs $\{(G_s^1, G_g^1), \ldots, (G_s^K, G_g^K)\}$, where $\mathcal{G}_s = \{G_s^1, \ldots, G_s^K\}$ and $\mathcal{G}_g = \{G_g^1, \ldots, G_g^K\}$ denote the corresponding witness set and grounded-graph set, respectively. For each pair (G_s^i, G_g^i), the graph G_g^i contains a k-grounded witness G_s^i under one or more original constraints corresponding to Σ_b. This holds due to the co-occurrence of matches of both c and P in G_g^i. The computation essentially repeats the steps below up to L times.

$$\mathcal{G}_s := \text{I-Chase}\ (\Sigma, G^L(v_t), 0); \qquad \mathcal{G}_g := \text{I-Chase}\ (\Sigma_b, \mathcal{G}_s, k).$$

Procedure L-Chase. (1) Unlike the fixed-point computation of I-Chase, L-Chase is invoked for up to L rounds within $G^L(v_t)$, starting from v_t and source constraints Σ_s whose pattern nodes match v_t, to avoid grounding with distant or irrelevant entities. (2) For each $\varphi = (P, c) \in \Sigma_s$, it generates candidate subgraphs $\mathcal{G}_c$ through an edge-growing graph pattern mining process and invokes VerifyWitness, yielding a set $\mathcal{G}_s$ of verified witnesses that are already 0-grounded in the local input graph.

Procedure L-Backchase. If $k > 0$ ("Backchase"), L-Backchase finds all inverse constraints $\varphi^{-1} = (c, P)$ whose consequent c has a match in G_s. For each, it inserts a minimal set ΔE of at most k new edges to complete a match of P under the same variable assignment, and constructs a connected graph G_g containing G_s and

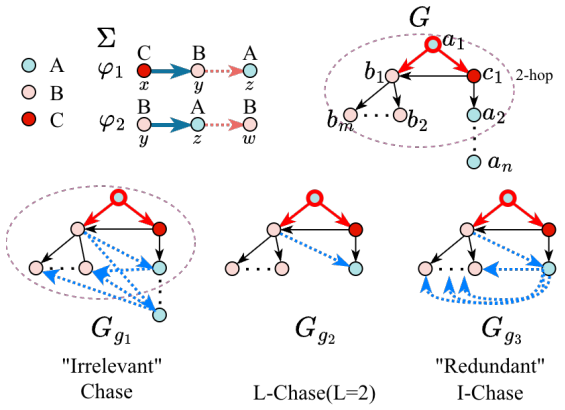


Fig. 3. Generating grounded witnesses (Inf-Chase); unlike I-Chase as a fixpoint computation, L-Chase reasons with Σ up to L times from a set of "source" constraints Σ_s.

ΔE. A candidate is retained only if $|\Delta E| \leq k$ and $G_g \models \varphi$, making G_s k-grounded by the original constraint $\varphi = (P, c)$. The procedure returns a set $\mathcal{G}_g$ of feasible grounded graphs and introduces at most $|\Sigma|k$ new grounded edges in total.

Example 7: Consider the constraint set $\Sigma = \{\varphi_1, \varphi_2\}$ shown in Fig. 3. All nodes a_i in G have type A; similarly, nodes b_i and c_i have types B and C, respectively. Constraint $\varphi_1 = (P_1, c_1)$ states that whenever a node of type B has an incoming edge from a node of type C, it should also have an outgoing edge to a node of type A. Constraint $\varphi_2 = (P_2, c_2)$ states that whenever a pattern $B \to A$ is present, a corresponding edge $A \to B$ should also exist. On the right-hand side of Fig. 3, the red bold edges denote a witness G_s rooted at the target node v_t, and the dashed oval marks its 2-hop neighborhood $G^2(v_t)$. We compare three outcomes.

(1) As P_1 is matched, a Chase, as a fixpoint process, enforces c_1 to insert edges between b_2 and any a node regardless of how large n is $(b_2 \to a_2, \ldots, a_n)$. Most of these edges fall outside of the receptive field of v_t and thus become irrelevant (see G_{g_1}).

(2) In contrast, I-Chase restricts reasoning to $G^2(v_t)$, so nodes outside the local neighborhood are excluded. However, after φ_2 is triggered, it may still generate multiple semantically equivalent local structures (e.g., $a_2 \to b_2, \ldots, b_m$), leading to an excessive number of over-grounded graphs that are mutually isomorphic (see G_{g_3}).

(3) L-Chase strikes a balanced alternative: since $L = 2$ and the witness G_s already covers the hop-1 neighborhood of v_t, one additional round of local reasoning within $G^L(v_t)$ is enough to produce the grounded result G_{g_2}. This avoids both the irrelevant edges of G_{g_1} and the redundant local structures of G_{g_3}. The example thus illustrates why bounded hop ranges and bounded reasoning rounds are preferable in practice for witness grounding. □

Correctness. We show that Inf-Chase correctly computes a set of grounded graphs $\mathcal{G}_g$ such that each $G_g \in \mathcal{G}_g$ contains a k-grounded witness G_s under some constraint $\varphi \in \Sigma$. We observe two invariants hold throughout the execution of Inf-Chase.

(1) The Chase phase identifies a set of subgraphs $\mathcal{G}_s$ of $G^L(v_t)$ such that, for each $G_s \in \mathcal{G}_s$, G_s already contains a co-occurring match of P and c for some $\varphi = (P, c) \in \Sigma$, and is therefore 0-grounded; and G_s has been verified by VerifyWitness to be either a factual or a counterfactual witness for $Q(M, v_t, G)$.

(2) For each witness $G_s \in \mathcal{G}_s$ derived in the Chase phase, there

Algorithm 1 ApxIChase

Input: Constraint set Σ, query $Q(M, v_t, G)$, integers L, K, and k;
Output: the current top-K grounded witness pair set W_K.

construct $G_\Sigma := \bigcup_{\varphi \in \Sigma}(P_\varphi \cup c_\varphi)$; initialize $\mathcal{M}$;
set $\mathcal{G}_s := \emptyset$, $\mathcal{C} := \emptyset$, $W_K := \emptyset$;
for $\ell = 1$ **to** L **do**
  $\Sigma_s := \{\varphi \mid u.l = v_t.l,\ u$ is from $\varphi.P,\ \varphi \in \Sigma\}$;
  $\mathcal{G}_s := \mathcal{G}_s \cup \mathsf{L-Chase}(\Sigma, \Sigma_s, G^L(v_t), G_\Sigma, \mathcal{M}, 0)$;
  for each $G_s \in \mathcal{G}_s$ **do**
    $\Sigma_b := \emptyset$;
    for each $\varphi \in \Sigma$ **do**
      if $\varphi.c$ has a match in G_s **then** $\Sigma_b := \Sigma_b \cup \{\varphi^{-1}\}$;
    $\mathcal{G}_g := \mathsf{L-Backchase}(\Sigma_b, G_s, G_\Sigma, \mathcal{M}, k)$;
    $\mathcal{C} := \mathcal{C} \cup \{(G_s, G_g) \mid G_g \in \mathcal{G}_g\}$;
    $W_K := \mathsf{UpdateWK}(\mathcal{C}, K)$;
return W_K

Fig. 4. Algorithm ApxIChase: bounded enforcement and edge insertion.

exists some constraint $\varphi = (P, c) \in \Sigma$ such that G_s already contains a match of the consequent c. The BackChase phase then enforces inverse constraint $\varphi^{-1} = (c, P)$ and applies $\mathsf{I-Chase}(\varphi^{-1}, G_s, k)$ to augment G_s under a local budget of k hypothesized edges for that match instance. If successful, the resulting graph G_g contains both the original match of c and a "recovered" match of P, therefore yields a k-grounded witness supported by φ. Repeating this process for applicable inverse constraints over all witnesses $G_s \in \mathcal{G}_s$ yields the grounded pairs returned by Inf-Chase.

Time Cost. Both Chase and backchase phases take $|\Sigma|$ calls of I-Chase, a fixed-point reasoning process as a sequence of L enforcement of constraints. It takes in total $O(|G^L(v_t)|^p)$ time to find matches of patterns from Σ, where p is the size of the largest pattern P a data constraint has in Σ. The inference cost is in $O(L|G^L(v_t)||N^L(v_t)|)$ time [26], [27], assuming $N^L(v_t)$ is much larger than the number of node features. Taken together, this yields a cost of Inf-Chase in $O(|G^L(v_t)|^p) + L|\Sigma||G^L(v_t)||N^L(v_t)|$.

While Inf-Chase can generate grounded witness pairs, (1) the resulting set may not optimize the overall quality objective; and (2) the exhaustive exploration of subgraphs in $G^L(v_t)$ remains a potential bottleneck. We next introduce two algorithms that specify Inf-Chase to efficient deployment for large graphs.

V. Online Witnesses Generation

Our first algorithm, denoted as ApxIChase and illustrated in Fig. 4, grows and maintains local candidate witnesses in a pool $\mathcal{C}$ from an online witness generation process, and constructs constraint-aligned ground graphs. To reduce matching and verification cost, it optimizes L-Chase with a shared pattern index and memoization map. At a request point, it efficiently selects top-K grounded witness pairs from $\mathcal{C}$, with a quality guarantee over generated grounded witness pairs.

Theorem 3: *Let $\mathcal{C}$ be the set of feasible grounded witness pairs generated by the online algorithm* ApxIChase *at a given request time.* ApxIChase *returns a top-K set $W_K \subseteq C$ such that $F(W_K) \geq (1 - \frac{1}{e}) \max_{P \subseteq C} F(P)$ ($|P| \leq K$), with a delay time in $O(|\Sigma|^2|G^L(v_t)| + L|\Sigma|^2 + |\Sigma||\mathcal{C}|K)$.* □

The quality and efficiency guarantee in theorem 3 is relative to the candidates available in $\mathcal{C}$ when the selection is invoked upon

Algorithm 2 Procedure L-Chase $(\Sigma, \Sigma_s, G^L(v_t), G_\Sigma, \mathcal{M}, 0)$

1: set $\mathcal{G}_c := \emptyset$; $\mathcal{G}_s := \emptyset$;
2: **for** $\varphi \in \Sigma_s$ **do**
3: graph $G_c :=$ getNext$(\varphi, G_\Sigma, \mathcal{M})$;
4: **if** $G_c \neq \emptyset$ **then**
5: $\mathcal{G}_c := \mathcal{G}_c \cup \{G_c\}$; update G_Σ, $\mathcal{M}$;
6: **if** VerifyWitness$(G_c, Q(M, v_t, G)) =$ true and $G_c \models \varphi$ **then**
7: $\mathcal{G}_s := \mathcal{G}_s \cup \{G_c\}$;
8: **return** $\mathcal{G}_s$;

Fig. 5. Procedure L-Chase

request. This allows ApxIChase to provide an anytime solution with a pragmatic relative quality guarantee.

Memoization Structures. ApxIChase stores and maintains the current top-K grounded witness pair set W_K together with a candidate pair pool $\mathcal{C}$. Besides, it also keeps track of two memoization structures below. (1) G_Σ is a graph pattern obtained as the union of the patterns $P \cup c$, encoding all the triple patterns from data constraints of Σ. Each edge $e = (u_s, u_p, u_o)$ in G_Σ carries a set of labels $\mathcal{L}(e) = \{i | \text{if } e \text{ is in } P_i \text{ or } c_i\}$ for any $\varphi_i = (P_i, c_i)$, *i.e.,* e indicates a common triple pattern in multiple constraints.

(2) A shared map $\mathcal{M}$ with size $|\Sigma| \times (p+2)$, with p the size of the largest pattern P in Σ, and for each entry $\mathcal{M}[i]$:

- $\mathcal{M}[i].\mathcal{P}$ is a size-$p+1$ binary vector, such that each slot $\mathcal{M}[i].\mathcal{P}[j]$ ($j \in [1, p]$) = 1 (resp. $\mathcal{M}[i].\mathcal{P}[p+1] = 1$) if and only if φ_i has its j-th triple pattern in P_i (resp. $\varphi_i.c_i$) has a non-empty edge match, in a current candidate subgraph G_s under verification; and
- $\mathcal{M}[i].b$, a budget number that records a lower bound of edges required to make G_s k-grounded by φ_i.

In addition, ApxIChase keeps a set of source constraints $\Sigma_s \subseteq \Sigma$ to start the enforcement in Chase phase; and for each witness G_s it derives a inverse constraint set $\Sigma_b \subseteq \Sigma^{-1}$ for the backchase phase.

Algorithm Outline. The main driving algorithm ApxIChase and the procedure L-Chase, L-Backchase are illustrated in Figs. 4 and 5, respectively; The detailed pseudocode of L-Backchase is deferred to [25]. ApxIChase first initializes the auxiliary structures above (lines 1–2). It then performs up to L rounds (line 3) of L-Chase (line 5) and, for each witness produced in the current round, invokes L-Backchase to construct the corresponding grounded graphs (line 10), consistently following Inf-Chase. The resulting grounded witness pairs are accumulated into the candidate pool $\mathcal{C}$ and then passed to UpdateWK, which, upon request time, greedily refreshes the current top-K grounded witness pairs under the set objective $F(\mathcal{P})$, thereby achieving an approximation ratio of $1 - \frac{1}{e}$ for the optimal K-set available at request time (see details of UpdateWK in [25]). The correctness and time cost follow from the counterpart of Inf-Chase; we present the detailed analysis in [25].

Optimization. ApxIChase uses several strategies with different focus. (1) To avoid irrelevant grounding that contributes little to $Q(M, v_t, G)$, procedure L-Chase calls a function getNext (line 3), which follows a prioritized traversal of G_Σ to prefer constraints whose pattern edges occur more frequently in Σ, and grows candidate subgraphs G_c accordingly. This favors common constraint patterns and reduces the exploration of rare or less representative ones. Other query selectivity or cardinality estimation techniques, e.g., [28], [29], are also readily applicable here.

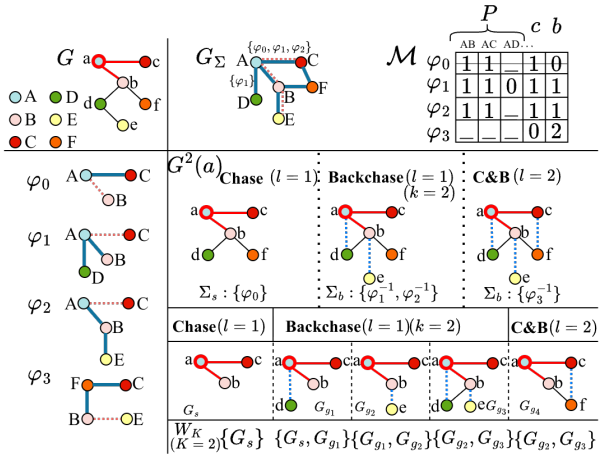


Fig. 6. ApxIChase: running of C& B in 2 rounds

(2) To reduce redundant inference cost, procedure VerifyWitness adopts an incremental inference strategy. Instead of restarting the inference process from scratch for every candidate (line 6 of VerifyWitness), it first performs a once-for-all inference for $Q(M, v_t, G^L(v_t))$ and caches intermediate node embeddings. When verifying a candidate G_c, it reuses cached results whenever possible and incrementally updates only the portion of message passing affected by G_c, thereby reducing repeated computation.

(3) Procedure L-Backchase reduces grounding cost by monitoring $\mathcal{M}$. In particular, it maintains a lower bound on $|\Delta E|$ as the number of missing edge matches, recorded by the number of 0 entries in $\mathcal{M}[i].\mathcal{P}$. The map $\mathcal{M}$ also enables early detection and runtime pruning for Σ_s and Σ_b by checking whether $\mathcal{M}[i].\mathcal{P}[p+1]$ (the consequent) already has a match.

(4) ApxIChase also copes with large constraint sets Σ by pruning constraints that are logically implied by others. We present the details in the appendix/full version [25].

These strategies are effective: as verified in our experimental study, they can improve the naive implementation naiveChase by *two orders of magnitude* in efficiency on average.

Example 8: Consider running ApxIChase on the example in Fig. 6. The original graph G (top-left) contains six nodes a, b, c, d, e, f, whose types are A, B, C, D, E, F, respectively, with four constraints $\Sigma = \{\varphi_0, \varphi_1, \varphi_2, \varphi_3\}$ shown on the left. Each constraint $\varphi_i = (P_i, c_i)$ is illustrated with its antecedent P_i (blue solid edges) and consequent c_i (pink dotted edge). The merged pattern G_Σ (top-right) encodes the union of $P_i \cup c_i$ for all $\varphi_i \in \Sigma$. The memoization map $\mathcal{M}$ records the grounding status of each constraint after one Chase round. For example, $\mathcal{M}[0].b = 0$ (as all the pattern edges have a match without "0" entries) indicates that $G_s \models_0 \varphi_0$, hence φ_0 is satisfied and can be removed from subsequent enforcement stage.

(1) During the first Chase phase, the receptive field of the target node a (*i.e.,* $G^2(a)$) has $\Sigma_s = \{\varphi_0\}$. In the first Backchase, given budget $k = 2$, the inverse constraint set $\Sigma_b = \{\varphi_1^{-1}, \varphi_2^{-1}\}$ is enforced.

(2) In the second round ($L = 2$), constraint φ_3^{-1} is also triggered, enforcing the new edge (c, f). The bottom row shows the evolution of the top-K window W_K (with $K = 2$), whose entries are grounded witness pairs; for visual simplicity, we display only their corresponding grounded graphs. Initially, G_s is discovered as a 0-grounded witness for φ_0. G_{g_1} and G_{g_2} represent $G_s \models_1 \varphi_1$ and $G_s \models_1 \varphi_2$, updating $W_K = \{G_{g_1}, G_{g_2}\}$. Upon the arrival of G_{g_3},

ApxIChase prefers G_{g_3} given its better coverage of both φ_1 and φ_2, and replaces G_{g_1}. The current top-K set becomes $\{G_{g_2}, G_{g_3}\}$.

(3) The process continues, and after another round of Chase & Backchase, a new graph G_{g_4} satisfies φ_3 ($G_s \models_1 \varphi_3$), yet due to a weak link to a, it does not improve the F-score ranking, hence W_K remains unchanged. The algorithm returns W_K as the result. □

VI. Fast Witnesses Generation

Our second algorithm specifies Inf-Chase for interpretable graph learning over large hierarchical or tree-like graphs, such as traffic, power delivery, epidemic, citation or referral networks [30]. In such scenarios, the grounded graphs are often close to a tree.

Grounding Witnesses as Trees. We consider a practical variant of our problem, where (1) each constraint $\varphi = (P, c)$ in Σ has a tree pattern $P \cup c$, (2) each witness $G_s \in \mathcal{G}_s$ is a weighted tree rooted at v_t, and (3) each grounded graph $G_g \in \mathcal{G}_g$ is a weighted arborescence of $G^L(v_t)$ rooted at v_t and containing G_s. This setting aligns G_g with the message-passing structure of the inference query $Q(M, v_t, G)$. A weight function $w(e)$ for an edge $(u, w) \in G$ can be introduced using representative edge scores adopted in GNN behavior and explainability analysis [5], such as embedding similarity, where $w(e) = \mathrm{sim}(h_u^{(L)}, h_w^{(L)}) + \varepsilon_e, \mathrm{sim}(a, b) = \frac{a^\top b}{\|a\| \, \|b\|}$, and ε_e is a small random noise (*e.g.,* Gumbel or Gaussian) that introduces diversity across m sampled trees. Other options such as resistance distance [31], Shapley value [10], or sensitivity [32] can also be applied. The problem remains computationally challenging in general due to the cost of witness discovery by graph pattern matching. We next introduce a fast heuristic, denoted as HeuIChase, to generate grounded graphs G_g as maximum weighted arborescences.

Outline. The algorithm follows the framework Inf-Chase with a L-Chase step and a L-Backchase step to generate grounded witnesses. Unlike ApxIChase, HeuIChase makes the following specifications.

(1) It uses a revised L-Chase, which generates witnesses G_s via tree pattern matching and verification between a tree pattern P and a set of m sampled trees from $G^L(v_t)$. (2) It extends each G_s to a weighted arborescence G_g rooted at v_t, which serves as a heuristic realization of L-Backchase. To ensure G_s is contained in G_g, it shrinks G_s to a "super node" and uses the contracted graph of $G^L(v_t)$ as input. This reduces the problem to computing a maximum-weight arborescence. Edmonds' algorithm [33] is then applied to derive G_g over the contracted graph. The procedure UpdateWK remains unchanged and maintains the top-K grounded witness pairs greedily.

Analysis. The algorithm HeuIChase correctly generates feasible grounded witness pairs in the tree/arborescence setting, following the framework of Inf-Chase and the correctness of Edmonds' algorithm. Its main cost saving comes from replacing general subgraph enumeration by tree matching and replacing explicit backchase repair by arborescence expansion. Treating the maximum tree-pattern size and the pruned rule-pool size as small constants, the cost up to request time can be summarized as $O\big(Lm|G^L(v_t)| + |\mathcal{C}||G^L(v_t)| \log |G^L(v_t)| + |\mathcal{C}|K\big)$. For HeuIChase, we set $\beta = 0$ and omit the explicit alignment term in the pairwise score used by UpdateWK. See [25] for the detailed derivation.

VII. Experimental Study

A. Experimental Settings

Using real-world and synthetic benchmark datasets, we evaluate the effectiveness, efficiency, and scalability of our algorithms.

Dataset	|V|	|E|	# attributes
BAShape [4]	2,020,000	12,055,704	1
Cora [30]	2,708	5,429	1,433
DBLP [36]	26,128	296,563	334
ATLAS [35]	59,148	40,823	64
MUTAG [34]	17.9(avg.)	19.8(avg.)	7
ogbn-Papers100M [37]	111M	1.6B	128
Tree-Cycles [4]	67M	1.4B	5

TABLE I
Summary of datasets

Datasets. We use five real-world graphs and two synthetic graphs for evaluation (summarized in Table I). (1) *MUTAG* [34], a set of 188 chemical compound graphs. Each graph has a class label indicating whether it is mutagenic or non-mutagenic. Node feature is a 7-dimensional one-hot encoding of atom types (*e.g.,* C, N, O). For this graph-level task, the model pools node embeddings to obtain a graph prediction; witness verification and fidelity use this graph-level output. (2) *ATLAS* [35] is a cyber event graph for intrusion detection. Nodes represent system entities (e.g., process, file, connection), and edges denote data flow or dependencies. (3) *Cora* [30] is a citation network with 7 categories. Each node has a 1,433-dimensional feature vector, and edges represent citation links among papers. (4) *DBLP* [36] is a heterogeneous bibliographic graph with typed nodes and relations, where nodes represent authors, papers, venues, and terms, and edges capture their interactions. We consider the task of author node classification. Since the prediction target is the author type, we report the author feature dimension (334) in Table I, while other node types use type-specific features. (5) *BAShape* [4] is a synthetic dataset generated by attaching house-shaped motifs to a Barabási–Albert base graph. A node label corresponds to structural roles (top, middle, bottom, or background). (6) *ogbn-Papers100M* [37] is a large-scale citation network with 1.6 billion citations (edges). Each paper has a 128-dimensional feature vector and is classified into one of 40 research fields. (7) *Tree-Cycles* [4] is a synthetic dataset with hierarchical graphs combining trees and cyclic structures, with 1.4 billion edges, and nodes in five categories represented by 5-dimensional one-hot features. Among these datasets, *DBLP* is used both in the overall efficiency & effectiveness evaluation and as the primary benchmark for detailed sensitivity analysis.

Constraint Generation. For each dataset, we derive candidate constraints from the training portion. For node-level datasets, constraints are constructed from local L-hop neighborhoods induced from labeled training nodes; for graph-level datasets, constraints are derived directly from the training graphs. We extract typed local patterns from these training-derived structures and instantiate constraints in the form $\varphi = (P, c)$, where P is the antecedent and c is a single-edge consequent. To simulate incomplete observations, we construct observed input graphs by masking a fraction of edges as a preprocessing step; the default mask ratio is 5%, and we vary it up to 20% in sensitivity studies. For DBLP, Cora, and MUTAG, the released implementation instantiates candidate rules from configured typed motifs, filters them by occurrence support, and ranks them to form a bounded constraint pool. It further filters candidates by consequent matchability on the selected workloads before masking; thus candidate construction from training data and workload-based selection are distinct steps. Constraint-pool sizes and pattern statistics are reported in the full version [25].

Graph Models. We use three representative GNN families in our experiments: GCNs [1], GATs [38], and GraphSAGE [39]. All models are implemented in PyTorch Geometric with a hidden dimension of 32, and the model depth follows the experimental setting.

We use the standard train/validation/test split associated with each dataset and configuration, and train each model for 200 epochs using Adam optimizer ($lr = 0.01$). All GNN models achieve competitive predictive performance, with test accuracy ranging from 82.4% to 91.1%. Our method is model-agnostic within the fixed, deterministic message-passing setting considered here: it verifies witnesses against the given pretrained model.

Algorithms. We implemented naiveChase, ApxIChase, and HeuIChase. These methods share the same top-k selection strategy, but employ different witness generation and grounding mechanisms. We compare them with two representative post-hoc GNN explainers whose explanation subgraphs can be extracted for a fair comparison. (1) *GNNExplainer* [4] is an established post-hoc method that identifies subgraphs most influential to the model's prediction by optimizing a continuous edge mask through gradient-based learning. (2) *PGExplainer* [5] is a learning-based method that learns a neural edge generator to produce explanations conditioned on node embeddings. Their generated subgraphs are treated as baseline witnesses for comparison. Unlike our methods, these baselines do not perform the subsequent G_s to G_g grounding step; their constraint coverage is therefore evaluated only under strict satisfaction of the generated subgraphs. In addition, to test scalability, we also implemented paraIChase, a parallelized implementation of Inf-Chase in shared memory, but by partitioning the inference query workload (see details in appendix [25]).

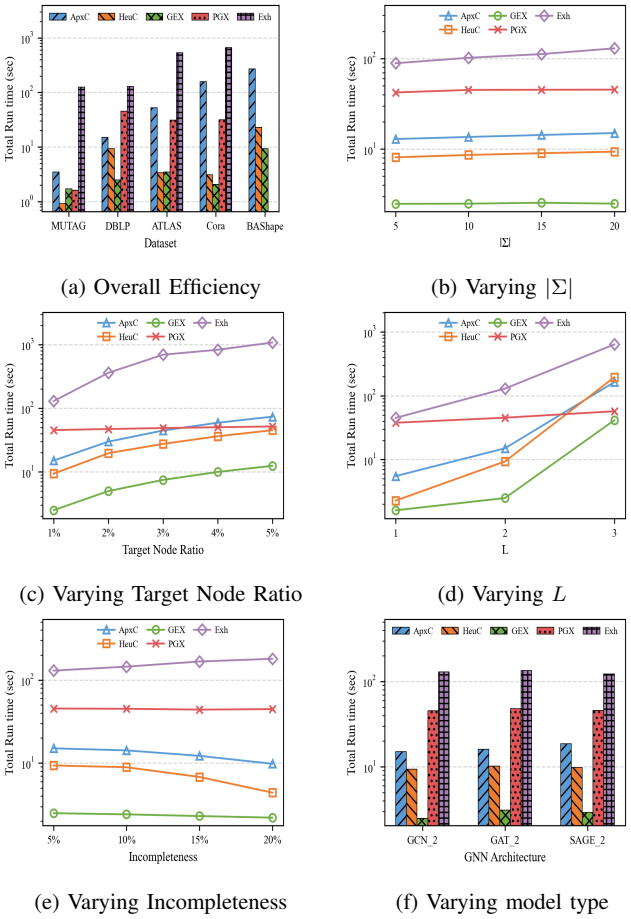
Evaluation Metrics. We report both the efficiency and effectiveness of all methods. (1) For *Efficiency*, we report the total runtime including candidate generation, verification, window maintenance, and grounding-related evaluation, until no further update can be made for our methods (e.g., ApxIChase, HeuIChase, and naiveChase). This runtime excludes the preprocessing time used to construct the shared input graph. For the baseline GNN explainers, we report the total runtime required to generate explanation subgraphs under the same input graph, model, and task setting. (2) For *Effectiveness*, we evaluate *Coverage*, *Conciseness*, and *Fidelity*. The coverage used in the optimization objective and conciseness are defined in Section III.

In particular, we report *normalized coverage* as the primary coverage metric. It focuses on the subset of constraints that are active for the current workload, namely those whose consequent can be matched and whose antecedent is already node-complete in the current input graph, and measures how many of them are successfully grounded within budget k. This reported measure uses the active subset as its denominator, whereas F uses the full constraint set Σ. The implementation reports zero when no constraint is active and averages the reported coverage over workloads. For fidelity, following conventional post-hoc explanation methods [4], [5], we report $1-$*fidelity-minus* so that higher values indicate less probability loss for the original predicted class:

$$\mathsf{Fid}(G_s) = 1 - \big(\Pr\big(Q(M, v_t, G)\big) - \Pr\big(Q(M, v_t, G_s)\big)\big)\,,$$

where $\Pr(\cdot)$ denotes the probability of the same class predicted on G in both terms. A score above 1 indicates that G_s increases this class probability. This metric is evaluated on G_s, rather than G_g.

Environments. All experiments are conducted on CWRU HPC cluster. Each compute node has an Intel Xeon Silver 4216 CPU at 2.10GHz (16 cores, 32 threads), 125GB of RAM, and a Tesla V100-SXM2 GPU with 32GB of memory. SLURM allocates access to a single node per test. Parallel variants utilize up to 20 CPU threads with GPU acceleration using NVIDIA driver version 560.35, and CUDA version 12.6. The code is available at [40].



(a) Overall Efficiency (b) Varying $|\Sigma|$

(c) Varying Target Node Ratio (d) Varying L

(e) Varying Incompleteness (f) Varying model type

Fig. 7. Performance Evaluation: Efficiency

B. Experimental Results

Exp-1: Efficiency. We compare the runtime of all methods on the benchmark datasets under their default model settings. For simplicity, we abbreviate the methods as follows: APXC (ApxIChase), HEUC (HeuIChase), GEX (GNNExplainer), PGX (PGExplainer), and EXH (exhaustive naiveChase). Unless explicitly varied, we use the default setting ($K = 6$, $k = 2$, $L = 2$, incompleteness $= 5\%$, $\alpha = \beta = \frac{1}{2}$, $\gamma = 1$). $|\Sigma|$ is dataset dependent [25]. For DBLP, the default setting is $|\Sigma| = 20$. For node-level datasets, the default sampled target ratio is $\frac{|V_t|}{|V|} = 1\%$, where *incompleteness* denotes the proportion of masked edges and $|V_t|$ the number of sampled test nodes.

Overall Efficiency. Fig. 7(a) reports the total runtime across datasets. Our methods substantially improve over naiveChase (Exh), whose exhaustive constraint enforcement incurs high overhead. For example, on Cora, ApxIChase and HeuIChase are about $4.2\times$ and $214\times$ faster than naiveChase, respectively; on MUTAG, the speedups are about $35.7\times$ and $137\times$. These gains come from bounded enforcement and more efficient candidate generation. Among our methods, HeuIChase is consistently the fastest due to its constrained search strategy, while ApxIChase incurs moderate overhead from more general exploration. Among the baselines, GNNExplainer (GEX) is the fastest, whereas PGExplainer (PGX) is more expensive due to global training and does not scale to large datasets such as BAShape.

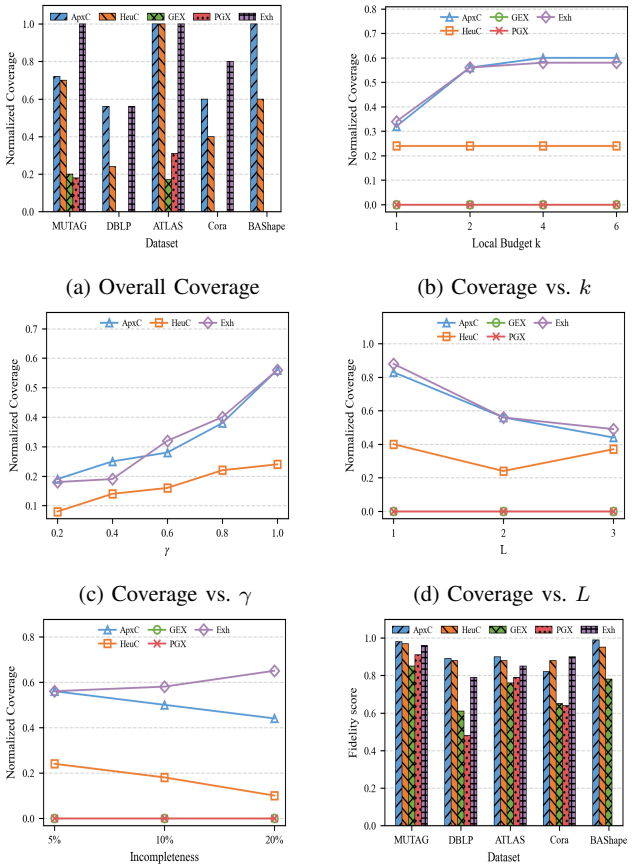
Impact of $|\Sigma|$. We vary the total number of data constraints $|\Sigma| \in \{5, 10, 15, 20\}$. As shown in Fig. 7(b), ApxIChase and HeuIChase exhibit only mild runtime growth as $|\Sigma|$ increases, since their cost is governed mainly by the number of activated constraints rather than the full constraint pool. This trend is gradual across the entire range: the runtime of ApxIChase increases from 12.9s to 15.03s, while that of HeuIChase increases from 8.1s to 9.35s. In contrast, naiveChase is highly sensitive to $|\Sigma|$, as more constraints directly increase the cost of exhaustive fixpoint enforcement. GNNExplainer and PGExplainer remain largely insensitive because they do not perform post-hoc constraint checking or grounding.

Varying the Size of Test Set $|V_t|$. We vary the proportion of target nodes from 1% to 5% on DBLP while fixing other parameters. As shown in Fig. 7(c), all methods become slower as $|V_t|$ grows, since each target node induces an independent workload. This trend is especially clear for ApxIChase, HeuIChase, and GNNExplainer: their runtimes increase from 15.03s to 73.7s, from 9.35s to 45.1s, and from 2.49s to 12.43s, respectively, as the target ratio grows from 1% to 5%. Both ApxIChase and HeuIChase scale approximately linearly, with HeuIChase consistently faster due to cheaper per-target processing. GNNExplainer also grows nearly linearly. By contrast, PGExplainer is less sensitive to $|V_t|$ because its dominant cost lies in shared training, while naiveChase grows the fastest due to repeated exhaustive enforcement for each target. At the same time, PGExplainer increases only modestly, from 45.33s to 51.7s, whereas naiveChase rises sharply from 129.7s to 1080.0s.

Varying GNN *Layer* L. We use GCN and vary the number of layers $L \in \{1, 2, 3\}$ on DBLP. As shown in Fig. 7(d), the runtime of all methods increases with L, since larger L induces larger local neighborhoods. HeuIChase remains relatively efficient, but its runtime still rises from 2.27s at $L = 1$ to 9.35s at $L = 2$, and to 195.8s at $L = 3$, slightly exceeding ApxIChase at the largest depth. This suggests that the Edmonds-based arborescence step can still become a bottleneck on large dense multi-hop neighborhoods, as the constructed arborescence may itself be too large and dense. ApxIChase and naiveChase also slow down with larger L, while PGExplainer remains relatively stable as its dominant cost comes from the shared explainer model rather than per-instance grounding.

Varying Incompleteness. We vary the degree of graph incompleteness from 5% to 20%. As shown in Fig. 7(e), the runtime of ApxIChase, HeuIChase, and GNNExplainer decreases as the input graph becomes sparser, since fewer observed structures remain for witness generation and grounding. This decrease is gradual but consistent: ApxIChase drops from 15.03s to 9.8s, HeuIChase from 9.35s to 4.4s, and GNNExplainer from 2.49s to 2.2s as incompleteness increases from 5% to 20%. In contrast, naiveChase becomes slower as incompleteness increases, because missing structure imposes a heavier exhaustive repair burden. PGExplainer remains largely stable, as its dominant cost comes from the shared explainer model rather than per-instance structural repair. Its runtime stays within a narrow range, from 45.33s to 44.6s, across all four settings.

Varying GNN *Model Type*. We evaluate all methods on DBLP using three representative backbones: 2-layer GCN, GAT, and GraphSAGE. As shown in Fig. 7(f), ApxIChase and HeuIChase remain relatively stable across different backbones, indicating that their cost is dominated more by candidate generation and grounding than by the backbone itself. In particular, ApxIChase shows only a mild increase from GCN to GAT and GraphSAGE, while HeuIChase remains in a similarly narrow range across all three settings. The same trend holds



(a) Overall Coverage (b) Coverage vs. k

(c) Coverage vs. γ (d) Coverage vs. L

(e) Coverage vs. Incompleteness (f) Overall Fidelity

Fig. 8. Performance Evaluation: Effectiveness

for GNNExplainer, PGExplainer, and naiveChase, all of which show only moderate variation across architectures. Although the absolute runtime changes slightly from one backbone to another, the gap between methods remains broadly consistent. Overall, the relative runtime ordering remains largely unchanged, indicating that runtime differences are driven mainly by algorithmic design rather than by the choice of GNN backbone.

Exp-2: Effectiveness. We next evaluate witness quality in terms of conciseness, coverage, and fidelity. Unless otherwise specified, all results use the default setting in Section VII-A. Overall effectiveness comparisons are reported across datasets, while detailed sensitivity analyses are conducted on *DBLP*.

Overall Coverage. Fig. 8(a) compares the normalized coverage across datasets. The grounding-aware methods achieve substantially higher coverage than the raw explanation baselines. On *ATLAS*, all three grounding methods achieve full coverage; on *MUTAG*, ApxIChase and HeuIChase remain high (0.72 and 0.70), while naiveChase reaches 1.0. Across datasets, ApxIChase is the most robust overall, e.g., on *DBLP* it clearly outperforms HeuIChase (0.56 vs. 0.24). Even naiveChase does not always achieve the best coverage, suggesting that more aggressive repair does not uniformly yield broader grounding. In contrast, GNNExplainer and PGExplainer generally achieve low coverage, since they do not perform grounding.

Effect of Grounding on Baseline Explainers. Table II reports the normalized coverage of GNNExplainer and PGExplainer before and

Dataset	GNNExplainer			PGExplainer		
	Before	After	Δ	Before	After	Δ
MUTAG	0.200	0.657	**+0.457**	0.180	0.571	**+0.391**
DBLP	0.000	0.240	**+0.240**	0.000	0.320	**+0.320**
ATLAS	0.170	0.540	**+0.370**	0.310	0.480	**+0.170**
Cora	0.000	0.596	**+0.596**	0.000	0.445	**+0.445**

TABLE II
EFFECT OF GROUNDING ON BASELINE EXPLAINERS. Δ DENOTES THE ABSOLUTE INCREASE IN NORMALIZED COVERAGE AFTER GROUNDING.

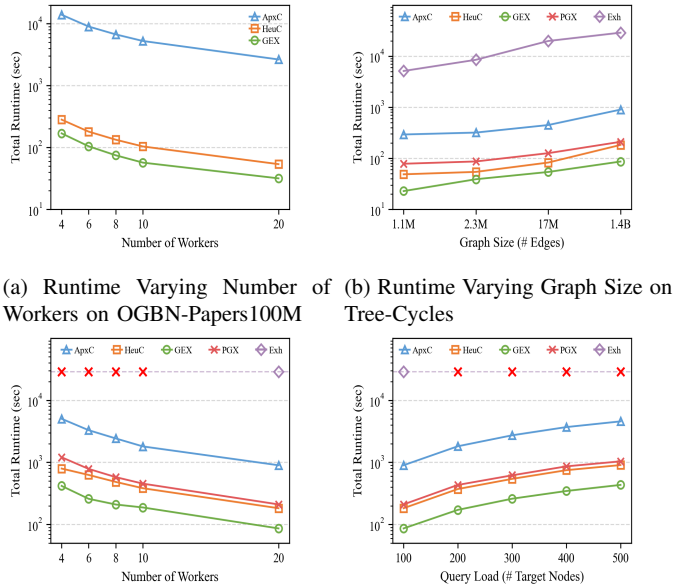
after applying our grounding process under the default experimental configuration. For example, coverage increases from 0.000 to 0.596 for GNNExplainer on Cora and from 0.000 to 0.320 for PGExplainer on DBLP. These gains show that grounding also benefits baseline witnesses, complementing the comparison with raw baseline explanations in Fig. 8(a).

Coverage vs. Budget k. We study how normalized coverage changes with the local budget $k \in \{1, 2, 4, 6\}$ on *DBLP*. As shown in Fig. 8(b), increasing k from 1 to 2 clearly improves coverage for ApxIChase and naiveChase, while further increases yield only marginal gains. This suggests that most active constraints that can be grounded by explicit repair already require only a small local budget, and that the remaining uncovered ones are mainly limited by structural mismatch rather than insufficient edge insertions. In contrast, HeuIChase remains flat across all values of k, consistent with its current heuristic design based on arborescence construction rather than an explicit k-sensitive L-Backchase step. The two post-hoc baselines, GNNExplainer and PGExplainer, stay at zero coverage throughout, since they do not perform the grounding step.

Coverage vs. γ. We study how normalized coverage changes as the weighting parameter $\gamma \in \{0.2, 0.4, 0.6, 0.8, 1\}$ varies on *DBLP*. As shown in Fig. 8(c), increasing γ improves coverage for all three grounding methods. This confirms that placing more emphasis on collective constraint coverage in the set-level objective helps top-K selection. The gain is most pronounced for ApxIChase and naiveChase, while HeuIChase also improves but more mildly, indicating that the objective can help select better candidates even when the candidate space is structurally constrained. Overall, this experiment shows that the set-level coverage term is effective in steering witness selection toward better-grounded results.

Coverage vs. L. We analyze how normalized coverage changes with model depth $L \in \{1, 2, 3\}$ on *DBLP*. As shown in Fig. 8(d), the normalized coverage of ApxIChase and naiveChase decreases as L increases. For example, ApxIChase drops from 0.83 at $L = 1$ to 0.56 at $L = 2$ and 0.44 at $L = 3$, while naiveChase decreases from 0.88 to 0.56 and then 0.49. This is because larger L induces much larger local neighborhoods and activates many more constraints, while the number of constraints that can be covered does not increase at the same rate. For HeuIChase, the trend is less regular, but its overall coverage remains below ApxIChase, reflecting that larger L-hop neighborhoods do not necessarily benefit tree-biased candidate generation. Its coverage changes from 0.40 to 0.24 and then slightly rebounds to 0.36, suggesting that the tree-biased search remains sensitive to how the local neighborhood expands as L grows.

Coverage vs. Incompleteness. We vary graph incompleteness from 5% to 20% missing edges on *DBLP*. As shown in Fig. 8(e), the normalized coverage of ApxIChase and HeuIChase decreases as the input graph becomes sparser, since fewer observed structures remain available for witness generation and grounding. This shows that effective grounding still depends strongly on the quality of the



(a) Runtime Varying Number of Workers on OGBN-Papers100M
(b) Runtime Varying Graph Size on Tree-Cycles
(c) Runtime Varying Number of Workers on Tree-Cycles
(d) Runtime Varying Query Load on Tree-Cycles

Fig. 9. Scalability Test

observed local evidence, rather than only on the ability to add missing edges. In contrast, naiveChase improves as incompleteness increases, indicating that exhaustive repair becomes relatively more effective when more structure is missing. GNNExplainer and PGExplainer remain at zero coverage throughout.

Overall Fidelity. Fig. 8(f) reports the overall Fid scores (higher is better) across datasets. ApxIChase and HeuIChase maintain high fidelity, showing that the witnesses selected by our methods still preserve the model's prediction behavior well. Their advantage is most evident on *MUTAG* and *BAShape*, where both methods achieve scores close to 1, suggesting that, when local patterns are more explicit, grounding-aware selection more easily identifies faithful witnesses. On *DBLP* and *ATLAS*, our methods also consistently outperform GNNExplainer and PGExplainer. Interestingly, naiveChase does not always achieve the best fidelity, even though it performs more aggressive repair. This suggests that stronger constraint enforcement does not necessarily lead to more faithful witnesses, and may bias witness selection toward structures that are less aligned with the model's decision process.

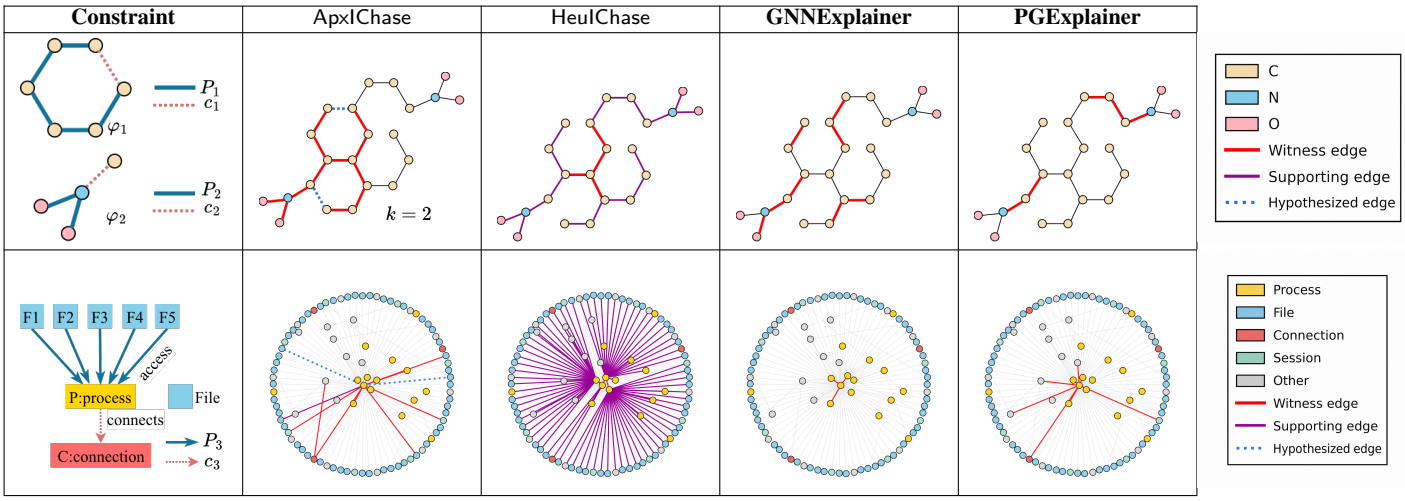
Exp-3: Scalability. We report the scalability of our parallel framework (paraIChase) using both real and synthetic graphs. In all the tests, L=2, k=2, K=6, and we use 100 target nodes by default.

OGBN-Papers100M. Figure 9(a) shows that both ApxIChase and HeuIChase achieve clear speedups when scaling from 4 to 20 workers. Despite highly unbalanced subgraph sizes (some >80K edges), runtimes remain tolerable—dropping from 13.9K→2.6K s for ApxIChase and 282 → 54 s for HeuIChase. PGExplainer and naiveChase fail to scale and do not complete within time limits. GNNExplainer remains fastest but saturates beyond 10 workers.

Tree-Cycles. We further evaluate scalability on the synthetic Tree-Cycles dataset under three settings.

(1) *Varying graph size:* As the graph grows from 1.1M to 1.4B edges (≈25 GB), both ApxIChase and HeuIChase show moderate runtime growth and remain tractable, while naiveChase exceeds 28,800 s (>8 h) with 20 workers and causes severe memory inflation. Each processor handles 4–6 L-hop subgraphs (≈4.6K edges). Under naiveChase,

TABLE III
CASE STUDY: APPLICATION OF GROUNDED WITNESS GENERATION FOR MOLECULAR PROPERTY ANALYSIS AND CYBERATTACK DETECTION.



enforcing all constraints can double or triple subgraph size, peaking at ≈1 GB per processor (≈20 GB total). In contrast, ApxIChase and HeuIChase only operate on compact witnesses—typically 20–40% of subgraph size—keeping per-processor memory below 200–300 MB. Overall, they reduce memory by 4–5× versus naiveChase while maintaining stable runtime and balanced load.

(2) *Varying processor count:* Fixing the $|E| = 1.4$B and 100 target nodes, we vary workers from 4 to 20 (Fig. 9(c)). Both ApxIChase and HeuIChase achieve sublinear yet consistent speedups; the measured makespan decreases by more than 5× between the tested worker counts, showing effective load balance and low communication contention under high parallelism.

(3) *Varying query load:* On the same graph with 20 workers, increasing target nodes from 100 to 500 (Fig. 9(d)) yields nearly linear runtime growth, dominated by extraction and grounding, with consistent trends across algorithms. On the reported large-graph workload, our methods achieve 1–2 orders of magnitude runtime improvement and 4–5× lower memory than the exhaustive naiveChase, maintaining stable scalability across graph size, worker count, and query load.

Summary. Overall, ApxIChase provides the strongest overall balance between computational efficiency, grounded coverage, and witness faithfulness, while HeuIChase remains suitable for fast grounded witness generation on large and tree-like graphs. Grounding quality depends on the supplied constraints; coverage and witness fidelity do not establish the correctness of inserted edges.

Exp-4: Case Studies. We next showcase how our methods link model predictions to local patterns specified by constraints.

Case I: Grounding Molecular Property Analysis. Table III (Row 1) shows a representative case from *MUTAG*. The leftmost column gives two constraints. Constraint φ_1 specifies a six-node ring pattern, where the antecedent P_1 is the solid blue path and the consequent c_1 is the missing ring-closing edge shown in red dotted style. Constraint φ_2 specifies a three-edge local functional pattern, where P_2 consists of the two solid blue edges and c_2 is the red dotted edge incident to the upper node. Starting from an incomplete molecular graph, ApxIChase produces a connected witness (red) that captures the branch pattern of φ_2, and uses two hypothesized edges (blue dotted) within budget k=2 to complete an additional ring structure. The dotted ring-closing edges are candidate relations to be validated by chemical evidence. HeuIChase instead yields a more tree-structured witness and expands it with observed "supporting edges" (purple) into a grounded graph supported by φ_2. GNNExplainer and PGExplainer recover only partial substructures from the left ring and the side branch, leaving the ring-closure constraints unsatisfied. The returned subgraphs fail to capture the complete constrained pattern.

Case II: Data Exfiltration Attack Investigation. We analyze a backdoor process flagged by a 3-layer GraphSAGE model for credential theft and data exfiltration. Table III (Row 2) compares interpretations under $\varphi_3 : P_3 \rightarrow c_3$, which associates access to five sensitive files—F1:/etc/passwd, F2:.ssh/id_rsa, F3:cookies.db, F4:confidential.pdf, and F5:credentials.txt—with the outbound connection C:198.51.100.25:443. ApxIChase yields a witness centered on the suspicious process, containing the outbound connection and three file-access edges. Backchase adds two hypothesized file-access edges (blue dotted) to complete P_3, yielding G_g that grounds φ_3. HeuIChase identifies the same central witness region and expands it with supporting edges (purple) into an arborescence grounding φ_3, without adding hypothesized edges. GNNExplainer returns only process-to-process access edges, matching neither c_3 nor any edge in P_3; PGExplainer covers c_3 but no edge in P_3. This case links model-verified witnesses to an investigative pattern, with the hypothesized file accesses requiring validation against audit logs or other evidence.

VIII. CONCLUSION

We studied constraint-grounded witnesses generation for explaining graph inference over incomplete graphs. We introduced a bi-level interpretation structure that pairs a model-verified witness with an auxiliary graph aligned with user-specified constraints. The auxiliary graph distinguishes observed context from hypothesized relations. We have developed a Chase-inspired framework, and two practical algorithms that use reverse constraint matching to generate candidate hypothesized edges, and to augment tree witnesses with weighted arborescence expansion. Our experiments on real-world and synthetic graphs demonstrate efficient witness generation, strong constraint coverage, and practical scalability while preserving model fidelity. Our case studies in molecular property analysis and cybersecurity also illustrate how the resulting grounded witnesses can be used to support subsequent domain investigation.

REFERENCES

[1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.

[2] T. Chen, D. Qiu, Y. Wu, A. Khan, X. Ke, and Y. Gao, "View-based explanations for graph neural networks," *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, pp. 1–27, 2024.

[3] V. Voinarovska, M. Kabeshov, D. Dudenko, S. Genheden, and I. V. Tetko, "When yield prediction does not yield prediction: an overview of the current challenges," *Journal of Chemical Information and Modeling*, vol. 64, no. 1, pp. 42–56, 2023.

[4] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "Gnnexplainer: Generating explanations for graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.

[5] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, and X. Zhang, "Parameterized explainer for graph neural network," *Advances in neural information processing systems*, vol. 33, pp. 19 620–19 631, 2020.

[6] M. Vu and M. T. Thai, "Pgm-explainer: Probabilistic graphical model explanations for graph neural networks," *Advances in neural information processing systems*, vol. 33, pp. 12 225–12 235, 2020.

[7] W. Lin, H. Lan, and B. Li, "Generative causal explanations for graph neural networks," in *International conference on machine learning*. PMLR, 2021, pp. 6666–6679.

[8] X. Wang, Y. Wu, A. Zhang, F. Feng, X. He, and T.-S. Chua, "Reinforced causal explainer for graph neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 2297–2309, 2022.

[9] W. Lin, H. Lan, H. Wang, and B. Li, "Orphicx: A causality-inspired latent variable model for interpreting graph neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 13 729–13 738.

[10] H. Yuan, H. Yu, J. Wang, K. Li, and S. Ji, "On explainability of graph neural networks via subgraph explorations," in *International conference on machine learning*. PMLR, 2021, pp. 12 241–12 252.

[11] B. Armgaan, M. Dalmia, S. Medya, and S. Ranu, "Graphtrail: Translating gnn predictions into human-interpretable logical rules," *Advances in Neural Information Processing Systems*, vol. 37, pp. 123 443–123 470, 2024.

[12] R. Kubo and D. Difallah, "Raw-explainer: Post-hoc explanations of graph neural networks on knowledge graphs," *arXiv preprint arXiv:2506.12558*, 2025.

[13] C. Geng, Z. Zhao, Z. Wang, H. Ye, Y. Jiang, and X. Si, "Logicxgnn: Grounded logical rules for explaining graph neural networks," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 4381–4412.

[14] J. Cheney, L. Chiticariu, W.-C. Tan *et al.*, "Provenance in databases: Why, how, and where," *Foundations and Trends in Databases*, 2009.

[15] Q. Song, M. H. Namaki, P. Lin, and Y. Wu, "Answering why-questions for subgraph queries," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 10, pp. 4636–4649, 2020.

[16] C. Zong and C. Zhang, "Effectively answering why questions on structural graph clustering," *Applied Soft Computing*, vol. 154, p. 111405, 2024.

[17] C. Zong, Z. Dong, X. Yang, B. Wang, T. Qiu, and H. Zhu, "Efficiently answering why-not questions on radius-bounded k-core searches," in *International Conference on Database Systems for Advanced Applications*. Springer, 2023, pp. 93–109.

[18] J. Du, S. Wang, H. Miao, and J. Zhang, "Multi-channel pooling graph neural networks." in *IJCAI*, 2021, pp. 1442–1448.

[19] W. Fan, X. Wang, Y. Wu, and J. Xu, "Association rules with graph patterns," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 8, no. 12, pp. 1502–1513, 2015.

[20] P. Pareti, G. Konstantinidis, T. J. Norman, and M. Şensoy, "Shacl constraints with inference rules," in *International Semantic Web Conference*. Springer, 2019, pp. 539–557.

[21] M. Calautti, M. Milani, and A. Pieris, "Semi-oblivious chase termination for linear existential rules: An experimental study," *Proc. VLDB Endow.*, vol. 16, no. 11, p. 2858–2870, 2023.

[22] M. J. Maher and D. Srivastava, "Chasing constrained tuple-generating dependencies," in *Proceedings of the fifteenth ACM SIGACT-SIGMOD-SIGART symposium on Principles of database systems*, 1996.

[23] A. Deutsch, L. Popa, and V. Tannen, "Chase & backchase: A method for query optimization with materialized views and integrity constraints," 2001.

[24] A. Lucic, M. A. Ter Hoeve, G. Tolomei, M. De Rijke, and F. Silvestri, "Cf-gnnexplainer: Counterfactual explanations for graph neural networks," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2022, pp. 4499–4511.

[25] "Full version," 2026. [Online]. Available: https://github.com/anthonyche/Grounded_Witness/blob/main/Grounding_GEXP_ICDE_2027_.pdf

[26] M. Chen, Z. Wei, B. Ding, Y. Li, Y. Yuan, X. Du, and J.-R. Wen, "Scalable graph neural networks via bidirectional propagation," *Advances in neural information processing systems*, vol. 33, pp. 14 556–14 566, 2020.

[27] H. Zhou, A. Srivastava, H. Zeng, R. Kannan, and V. Prasanna, "Accelerating large scale real-time gnn inference using channel pruning," *arXiv preprint arXiv:2105.04528*, 2021.

[28] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, L. W. Tan, K. Zeng, G. Cong, Y. Qin, A. Pfadler, Z. Qian, J. Zhou, J. Li, and B. Cui, "Cardinality estimation in dbms: A comprehensive benchmark evaluation," *Proceedings of the VLDB Endowment*, vol. 15, no. 4, pp. 752–765, 2022.

[29] Y. Sheng, X. Cao, K. Zhao, Y. Fang, J. Qi, W. Zhang, and C. S. Jensen, "Ace: A cardinality estimator for set-valued queries," *Proceedings of the VLDB Endowment*, vol. 18, no. 7, pp. 2112–2125, 2025.

[30] A. K. McCallum, K. Nigam, J. Rennie, and K. Seymore, "Automating the construction of internet portals with machine learning," *Information Retrieval*, 2000.

[31] M. Zhu, L. Zhu, H. Li, W. Li, and Z. Zhang, "Resistance distances in directed graphs: Definitions, properties, and applications," *Theoretical Computer Science*, vol. 1009, p. 114700, 2024.

[32] D. J. Klein, "Centrality measure in graphs," *Journal of mathematical chemistry*, vol. 47, no. 4, pp. 1209–1223, 2010.

[33] J. Edmonds *et al.*, "Optimum branchings," *Journal of Research of the national Bureau of Standards B*, vol. 71, no. 4, pp. 233–240, 1967.

[34] A. K. Debnath, R. L. Lopez de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, "Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. correlation with molecular orbital energies and hydrophobicity," *Journal of medicinal chemistry*, vol. 34, no. 2, pp. 786–797, 1991.

[35] A. Alsaheel, Y. Nan, S. Ma, L. Yu, G. Walkup, Z. B. Celik, X. Zhang, and D. Xu, "ATLAS: A sequence-based learning approach for attack investigation," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 3005–3022.

[36] X. Fu, J. Zhang, Z. Meng, and I. King, "Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding," in *Proceedings of the Web Conference 2020*, 2020, pp. 2331–2341.

[37] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," *NeurIPS*, 2020.

[38] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio *et al.*, "Graph attention networks," *stat*, 2017.

[39] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *NeurIPS*, 2017.

[40] "Code repository," 2026. [Online]. Available: https://github.com/anthonyche/Grounded_Witness/tree/main

[41] W. Fan, Y. Wu, and J. Xu, "Functional dependencies for graphs," in *Proceedings of the International Conference on Management of Data (SIGMOD)*, 2016, pp. 1843–1857.

[42] L. C. Shimomura, G. Fletcher, and N. Yakovets, "Ggds: graph generating dependencies," in *CIKM*, 2020.

[43] R. M. Karp, "Reducibility among combinatorial problems," in *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*. Springer, 2009, pp. 219–241.

[44] W.-S. Han, J. Lee, and J.-H. Lee, "Turboiso: towards ultrafast and robust subgraph isomorphism search in large graph databases," in *Proceedings of the International Conference on Management of Data (SIGMOD)*, 2013, pp. 337–348.

[45] M. Han, H. Kim, G. Gu, K. Park, and W.-S. Han, "Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together," in *Proceedings of the International Conference on Management of Data (SIGMOD)*, 2019, pp. 1429–1446.

[46] D. Mawhirter, S. Reinehr, C. Holmes, T. Liu, and B. Wu, "Graphzero: A high-performance subgraph matching system," *ACM SIGOPS Operating Systems Review*, vol. 55, no. 1, pp. 21–37, 2021.

[47] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—i," *Mathematical programming*, vol. 14, no. 1, pp. 265–294, 1978.

IX. Appendix

A. Appendix A: Formal Notations and Proofs

Graph Data Constraints: Specifications. Graph functional dependencies [41], graph association rules [19], graph generation dependencies [42], and SHACL [20] provide sources of structural and value-binding requirements for graph data. We consider requirements expressed in the restricted form (P, c) of Section II, where P is a conjunction of triple patterns and c is a single triple pattern. Their use here is for local witness grounding: G_g contains a co-occurring match of P and c under the same assignment. This local condition does not establish general expressive or semantic equivalence with the dependency languages above.

Verification and Existence Complexity. We distinguish verification of a model witness from the complexity of finding a constraint-pattern match. For a fixed deterministic message-passing model and an explicitly supplied G_s, the factual or counterfactual witness test uses a constant number of model inferences. The input pattern size is a separate source of computational complexity.

Lemma 4: *The witness verification problem is in* PTIME. □

Proof. This result can be verified by constructing a PTIME procedure that invokes the inference process of the fixed graph ML model M, and checks by definition if G_s is either factual or counterfactual via two inference calls, in polynomial time [26]. □

We next analyze constraint-pattern satisfaction for a supplied grounded graph. The input specifies the observed graph G, a verified witness G_s, an auxiliary graph $G_g \supseteq G_s$, one constraint $\varphi = (P, c)$, and a bound k. The decision test checks whether the added edges satisfy $|E(G_g) \setminus E(G)| \leq k$ and whether $G_g \models \varphi$.

Lemma 5: *Checking existential constraint co-occurrence for a supplied grounded graph is NP-complete in combined graph-and-pattern size, even for one constraint and $k = 0$.* □

Proof. Membership in NP follows by guessing a homomorphism from the pattern $P \cup \{c\}$ to G_g and checking its mapped edges and typed nodes. The input also permits polynomial checks of witness verification for fixed M, graph containment, connectivity, and the edge-insertion budget. For hardness, start from a loop-free *Clique* instance (H, r) and build a connected graph G_g by adding an anchor a adjacent to every vertex of H and a target v_t adjacent to a. Choose the singleton $G_s = \{v_t\}$ and a fixed constant-output GNN, so G_s is verified. Set $k = 0$. Create a typed co-occurrence pattern with r clique variables of the type used in H, one anchor variable of a distinct type, an anchor-to-clique edge, and all clique edges; choose one clique edge as the consequent c. The remaining edges form P. A homomorphism of this pattern into G_g exists precisely when H contains an r-clique: the pattern's pairwise adjacent variables must map to distinct loop-free vertices. Thus the test is NP-hard even for one constraint and zero inserted edges. Combined with NP membership, the claim follows. The reduction uses existential pattern matching, consistent with Section II. □

Hardness of Witness generation. We next prove Theorem 1.

Proof. We reduce from the classical *Clique* problem [43]. Given an undirected graph $H = (V_H, E_H)$ and an integer $r \geq 3$, the question is whether H contains a clique of size r.

From this clique instance (H, r), we construct an instance of the decision version of witness generation as follows. The graph G contains a copy of H, where every copied node has the same type C. We then add two fresh nodes: a target node v_t of type T, and an anchor node a of type A. We add the edge (v_t, a), and for every node $u \in V_H$, we add the edge (a, u). If the graph model is directed, each undirected edge of H is replaced by two opposite directed edges; this does not affect the reduction.

We fix a deterministic GNN model M_0 that returns the same output for every input graph containing v_t. Hence, the singleton graph G_s consisting only of v_t is a factual witness for $Q(M_0, v_t, G)$. Therefore, the witness requirement is trivial, and the hardness comes entirely from grounding.

We use a single constraint $\Sigma = \{\varphi\}$, where $\varphi = (P, c)$. Let y be a variable of type A, and let $x_1, \ldots, x_r$ be variables of type C. The antecedent P contains (i) the anchor edge (y, x_1), and (ii) all clique edges (x_i, x_j) for $1 \leq i < j \leq r$, except for the pair (x_1, x_r). The consequent is exactly the missing edge $c = (x_1, x_r)$.

In the constructed instance, a grounded graph G_g satisfies φ if and only if it contains a match of an r-clique in the copy of H, together with the anchor edge from a to one clique node.

We now prove the correctness of the reduction.

($\Rightarrow$) Suppose that H contains a clique $\{u_1, \ldots, u_r\}$ of size r. Let G_g be the connected subgraph of G induced by $\{v_t, a, u_1, \ldots, u_r\}$. Under the homomorphism $h(y) = a$ and $h(x_i) = u_i$ for $1 \leq i \leq r$, every edge of P and the consequent edge c is present in G_g. Hence $G_g \models \varphi$. Since $k = 0$, no hypothesized edge needs to be added. Therefore, (G_s, G_g) is a feasible grounded witness pair with $K = 1$.

($\Leftarrow$) Conversely, suppose that there exists a feasible grounded witness pair (G'_s, G_g) with $K = 1$, $k = 0$, and $G_g \models \varphi$. Since y has type A, and a is the only node of type A in G, the match of y must be a. Likewise, since $x_1, \ldots, x_r$ all have type C, they must be mapped to nodes in the copy of H. Because all edges in P and the consequent edge c are present in G_g, the images of $x_1, \ldots, x_r$ are pairwise adjacent in the copy of H, and thus form a clique of size r. Moreover, these images must be distinct: if two variables were mapped to the same node, then some required edge in P or the consequent c would become a self-loop, but H has no self-loops.

Hence, the constructed instance admits a feasible grounded witness pair if and only if H contains a clique of size r. Moreover, the construction is polynomial in the size of (H, r). Therefore, the decision version of the witness generation problem is NP-hard, even when M is fixed, $K = 1$, $k = 0$, and $|\Sigma| = 1$. □

Termination of I-Chase. We prove Lemma 2 under its stated active-domain assumption. Fix a finite vertex set V and a finite set of relation labels R. The search may insert a missing labeled edge between existing vertices. There are at most $|R|\,|V|^2$ possible labeled directed edges. Every effective insertion strictly increases the edge set. Consequently, an execution with no deletions and no new vertices terminates after at most $|R|\,|V|^2 - |E(G)|$ successful insertions. A terminal graph is saturated relative to the grounding assignments examined by the algorithm. Different choices for a consequent variable can produce different terminal graphs. For example, suppose the antecedent identifies a node a and the consequent requests an outgoing edge to an eligible node of type B. With two eligible nodes b_1, b_2, one run may insert (a, b_1) and another (a, b_2). If an existential match suffices for enforcement, both runs terminate, and the resulting graphs differ. Therefore termination holds without a confluence guarantee. A unique least fixed point requires a fixed exhaustive ground-instance semantics or another specified canonical insertion policy. The present framework uses bounded local search and does not rely on that stronger condition.

Algorithm 3 Procedure L-Backchase $(\Sigma_b, G_s, G_\Sigma, \mathcal{M}, k)$

set $\mathcal{G}_g := \emptyset$;
for each $\varphi^{-1} \in \Sigma_b$ **do**
  if $\mathcal{M}[i].b > k$ **then**
    continue;
  derive ΔE from $\mathcal{M}[i].P$ ("0" entries);
  construct and verify G_g (w. Weisfeiler-Leman test);
  if $|\Delta E| \leq k$ and $G_g \models \varphi_i$ **then**
    $\mathcal{G}_g := \mathcal{G}_g \cup \{G_g\}$;
return $\mathcal{G}_g$;

Fig. 10. Procedure L-Backchase

Lemma 6: *A bounded candidate-selection execution of* L-Chase *has no general confluence guarantee.* □

Proof. Consider a one-round selection policy with two eligible constraints whose patterns lead to distinct candidate extensions. If getNext selects one candidate in a round, processing one source constraint first can return a different candidate set from processing the other first. The procedure stops after its prescribed bound and need not explore both branches. Thus its candidate set need not be independent of processing order. This establishes the absence of a general confluence guarantee; it does not imply that every run yields different outputs. □

Axiomatic Analysis. We prove the three properties of the set objective $F(\mathcal{P})$ stated in Section III. Let $\mathcal{C}$ be the finite set of feasible grounded witness pairs for a fixed inference query $Q(M, v_t, G)$, budget k, and constraint set Σ. Since $|\mathcal{P}| \leq K$ and $\mathcal{C}$ is finite, there exists an optimal witness set $\mathcal{P}^* \in \arg\max_{\mathcal{P} \subseteq \mathcal{C}, |\mathcal{P}| \leq K} F(\mathcal{P})$.

(1) **Scale invariance**. Let $c > 0$ be a constant, and define $F_c(\mathcal{P}) = c \cdot F(\mathcal{P}) = c \cdot \sum_{(G_s, G_g) \in \mathcal{P}} q(G_s, G_g) + c \cdot \gamma \cdot \mathsf{cov}(\mathcal{P})$. For any two feasible witness sets $\mathcal{P}_1, \mathcal{P}_2 \subseteq \mathcal{C}$, $F_c(\mathcal{P}_1) \geq F_c(\mathcal{P}_2) \iff c \cdot F(\mathcal{P}_1) \geq c \cdot F(\mathcal{P}_2) \iff F(\mathcal{P}_1) \geq F(\mathcal{P}_2)$. Hence the maximizer of the objective remains unchanged under a common positive scaling of the component scores and the coverage term. Therefore, $\mathcal{P}^*$ is invariant to such normalization of user preference weights.

(2) **Non-monotonicity with respect to witness / grounding size**. This property concerns the sizes of individual witnesses or grounded graphs, rather than set inclusion over witness sets. Consider two singleton witness sets $\mathcal{P}_1 = \{(G_s, G_g)\}, \mathcal{P}_2 = \{(G'_s, G'_g)\}$. It is possible that $|G_s| \geq |G'_s|$ and yet $F(\mathcal{P}_1) < F(\mathcal{P}_2)$: for example, the larger witness G_s may have strictly lower conciseness, while yielding no compensating gain in alignment or coverage. Conversely, it is also possible that $|G_s| \geq |G'_s|$ and $F(\mathcal{P}_1) > F(\mathcal{P}_2)$: for example, the larger grounded graph G_g may ground additional constraints or require proportionally fewer inserted edges, thereby increasing the alignment and/or coverage contribution. Hence there is no monotone relationship between the sizes of individual witnesses / grounded graphs and the objective value $F(\mathcal{P})$.

(3) **Locality**. For any selected witness set $\mathcal{P}$, the contribution of each pair $(G_s, G_g) \in \mathcal{P}$ to the objective is captured locally by $q(G_s, G_g)$, which depends only on that witness and its corresponding grounded graph. The only interaction among different selected pairs arises through the set-level coverage term $\mathsf{cov}(\mathcal{P}) = \frac{|\mathsf{CovSet}(\mathcal{P})|}{|\Sigma|}$, $\mathsf{CovSet}(\mathcal{P}) = \bigcup_{(G_s, G_g) \in \mathcal{P}} \Sigma'(G_g)$. Therefore, no information outside the grounded witness pairs in $\mathcal{P}$ and their grounded graphs affects the value of $F(\mathcal{P})$. This establishes locality property.

Algorithm 4 Procedure VerifyWitness $(G_s, Q(M, v_t, G))$

Input: Candidate G_s containing v_t; query $Q(M, v_t, G)$.
Output: Whether G_s is a witness.
retrieve cached $y = Q(M, v_t, G)$;
if $Q(M, v_t, G_s) = y$ **then**
  return true;
return $(Q(M, v_t, G \setminus G_s) \neq y)$;

Fig. 11. Procedure VerifyWitness

Algorithm 5 : **Procedure** UpdateWK $(\mathcal{C}, K)$

$W_K \leftarrow \emptyset$
for $i = 1$ to K **do**
  for each $(G_s, G_g) \in \mathcal{C} \setminus W_K$, compute $\Delta((G_s, G_g) \mid W_K)$
  select $(G_s^*, G_g^*) \in \mathcal{C} \setminus W_K$ with the largest $\Delta((G_s, G_g) \mid W_K)$
  if no such pair exists **then**
    break
  $W_K \leftarrow W_K \cup \{(G_s^*, G_g^*)\}$
return W_K

Fig. 12. Procedure UpdateWK

Remark. The non-monotonicity above concerns changes in the sizes of individual witnesses or grounded graphs. It does not contradict the monotonicity of $F(\mathcal{P})$ as a set function with respect to set inclusion, which is the property used in approximability analysis.

B. Appendix B: Algorithms and Analysis

Procedure VerifyWitness. Procedure VerifyWitness (Fig. 11) accepts G_s if it preserves the original prediction (factual) or its removal changes that prediction (counterfactual). The reference prediction on G is cached, so each candidate requires at most two additional inference calls.

Implementation of naiveChase. The algorithm naiveChase enforce each graph constraint $\varphi \in \Sigma$ in $G^L(v_t)$.

(1) The I-Chase procedure extends Chase algorithm to perform a fixpoint enforcement of Σ as follows. Whenever there is an applicable constraint $\varphi = (P, c)$, it iteratively retrieves a subgraph $G_\varphi = G_s \oplus e_c$, where G_s is a subgraph of G that matches P (obtained by invoking fast subgraph match enumeration algorithms *e.g.,* [44]–[46]), and e_c is a single edge that matches the consequent (triple pattern) c, either inserted (enforced), or originally exists in G.

(2) For each such subgraph $G_\varphi = G_s \oplus e_c$, it invokes a verification process VerifyWitness, which performs the inference of M over G_φ to test whether G_φ is a witness for the inference query $Q(M, v_t, G)$. If so, it adds G_φ as a 0-grounded witness by φ in $G^L(v_t)$. For each $\varphi \in \Sigma$, it enumerates all candidate 0-grounded witness pairs. It then returns a top-K set that maximizes the overall objective $F(\mathcal{P})$.

Procedure L-Backchase **(line 10, Algorithm 1, Fig. 4)**. Given an inverse constraint set Σ_b, a verified witness G_s, the merged pattern graph G_Σ, the memoization map $\mathcal{M}$, and a budget k, procedure L-Backchase constructs the grounded graph set $\mathcal{G}_g$ used in the Backchase phase. For each applicable inverse constraint $\varphi^{-1} = (c, P) \in \Sigma_b$, it first checks whether the lower-bound budget $\mathcal{M}[i].b$

exceeds k. If not, it derives the missing hypothesized edges ΔE from the unmatched entries of $\mathcal{M}[i].P$, constructs a candidate grounded graph G_g, and verifies that $|\Delta E| \leq k$ and that P_i and the matched c_i co-occur under the same variable assignment. Each successful graph is added to $\mathcal{G}_g$, which is then returned to Algorithm 1 to form candidate pairs.

Procedure UpdateWK **(line 12, Algorithm 1, Fig. 4)**. Given the candidate pair pool $\mathcal{C}$ of grounded witness pairs generated so far, UpdateWK greedily refreshes the current top-K set under the set objective $F(\mathcal{P})$. At each step, for every candidate pair $(G_s, G_g) \in \mathcal{C} \setminus W_K$, it computes the marginal gain

$$\Delta((G_s, G_g) \mid W_K) = q(G_s, G_g) + \gamma \cdot \frac{|\Sigma'(G_g) \setminus \mathsf{CovSet}(W_K)|}{|\Sigma|},$$

and selects the pair with the largest marginal gain to update W_K. Since each iteration inserts at most one previously unselected pair and the procedure performs at most K iterations, the returned window always satisfies $|W_K| \leq K$. Hence, UpdateWK returns, upon request time, a greedy approximation to the best size-K set under $F(\mathcal{P})$ over the grounded witness pairs generated so far.

Optimization: Pruning Σ**.** To reduce the redundant constraint enforcement, ApxIChase applies a structural pruning rule over Σ to avoid repeatedly checking the subset of data constraints whose antecedents are subsumed by others.

Lemma 7: *Given two constraints $\varphi_i = (P_i, c)$ and $\varphi_j = (P_j, c)$ with the same consequent c, if $P_i \subseteq P_j$, then $G_s \models_k \varphi_j \Rightarrow G_s \models_k \varphi_i$.* □

A match of φ_j also certifies a match of φ_i. This supports skipping duplicate pattern checks. Coverage is still evaluated over the original Σ; pruning must preserve the identifier of every implied covered constraint. Removing φ_i from the denominator would change the optimization objective.

Proof sketch: Suppose $G_s \models_k \varphi_j$. Then, by definition, there exists a grounded graph $G_g = G_s \oplus \Delta E$ with $|\Delta E| \leq k$ such that $G_g \models \varphi_j$. Hence there exists a homomorphism h such that $h(P_j) \subseteq G_g$ and $h(c) \in E(G_g)$. Since $P_i \subseteq P_j$, the restriction of h to P_i also yields a match of P_i in G_g, while the same edge $h(c)$ still witnesses the consequent. Therefore $G_g \models \varphi_i$, and thus $G_s \models_k \varphi_i$. □

Analysis of ApxIChase. We prove that ApxIChase terminates and returns a feasible top-K grounded witness pair set as a $(1 - 1/e)$-approximation to the current optimum over the grounded witness pairs generated and available at the request time.

Correctness. The algorithm executes at most L rounds(line 3). In each round $l \in [1, L]$, L-Chase (line 6) performs bounded pattern matching over $G^L(v_t)$ without edge insertion, terminating when all source constraints Σ_s are processed. L-Backchase (line 12) enforces at most k new edges per applicable inverse constraint $\varphi^{-1} \in \Sigma_b$, as tracked by budget $\mathcal{M}[i].b$(Figure 10, line 3), since $G^L(v_t)$ is finite and each edge insertion reduces the available budget, the backchase phase terminates after at most $|\Sigma| \cdot k$ edge insertions. The outer loop completes in bounded steps and guarantees termination.

Every admitted pair must satisfy two checks. First, VerifyWitness confirms that G_s meets the chosen factual or counterfactual model test. Second, the backchase validation checks $|\Delta E| \leq k$ for the selected constraint match and verifies a co-occurring match of P and c in G_g. Together, these checks certify feasibility of the pair for that match. The budget map provides a pruning lower bound on missing edges.

Approximability. Let $\mathcal{C}$ denote the candidate pair pool of grounded witness pairs generated so far. Upon request time, UpdateWK greedily refreshes the current top-K set by selecting witness pairs from $\mathcal{C}$ according to their marginal gain under the set objective $F(\mathcal{P})$. This yields a $(1 - 1/e)$-approximation guarantee under our set objective, formalized in Theorem 3. We next prove Theorem 3

Proof. At any request time, let $\mathcal{C}$ denote the set of candidate grounded witness pairs generated so far, where each element is a pair (G_s, G_g). Consider the cardinality-constrained maximization problem $\max_{\substack{\mathcal{P} \subseteq \mathcal{C} \\ |\mathcal{P}| \leq K}} F(\mathcal{P})$, where, by definition, $F(\mathcal{P}) = \sum_{(G_s, G_g) \in \mathcal{P}} q(G_s, G_g) + \gamma \cdot \mathsf{cov}(\mathcal{P})$. We show that $F(\cdot)$ is a monotone submodular set function on the candidate domain with nonnegative pair scores $q(G_s, G_g)$. First, the term $\sum_{(G_s, G_g) \in \mathcal{P}} q(G_s, G_g)$ is modular, since it is a sum of fixed per-witness scores that depend only on individual pairs. Second, the term $\mathsf{cov}(\mathcal{P}) = \frac{\left|\bigcup_{(G_s, G_g) \in \mathcal{P}} \Sigma'(G_g)\right|}{|\Sigma|}$ is a normalized set-coverage function, where $\Sigma'(G_g) \subseteq \Sigma$ denotes the subset of constraints grounded by G_g.

Since set coverage is monotone submodular, $\mathsf{cov}(\mathcal{P})$ is monotone submodular as well. Therefore, $F(\mathcal{P})$, being the sum of a modular function and a monotone submodular function, is itself monotone submodular. Under the cardinality constraint $|\mathcal{P}| \leq K$, the standard result of Nemhauser et al. [47] implies that the greedy algorithm achieves a $(1-1/e)$-approximation to the optimum. Since UpdateWK constructs W_K by repeatedly selecting the grounded witness pair with the largest marginal gain under $F(\cdot)$ from the candidate pool generated so far, at any request time it returns a set W_K such that $F(W_K) \geq (1 - 1/e) \cdot F(\mathcal{P}^*)$, where $\mathcal{P}^* \in \arg\max_{\substack{\mathcal{P} \subseteq \mathcal{C} \\ |\mathcal{P}| \leq K}} F(\mathcal{P})$. Hence, UpdateWK achieves a $(1 - 1/e)$-approximation ratio with respect to the current optimum obtainable from the grounded witness pairs generated so far. □

Cost Analysis. For a fixed candidate pool $\mathcal{C}$, greedy selection performs at most K iterations. Each scans at most $|\mathcal{C}|$ candidates, and each coverage increment can be computed by inspecting at most $|\Sigma|$ constraint identifiers. Thus selection costs $O(K|\mathcal{C}||\Sigma|)$ under direct set representation. This proves the selection-cost part of Theorem 3.

Candidate generation has additional costs: graph-pattern matching, calls to the fixed GNN for witness verification, and construction of proposed edges. The number of generated candidates depends on the behavior of getNext and on the selected patterns. We make no worst-case end-to-end polynomial-delay claim for this search. Let p denote the largest number of triples in an input antecedent. In the worst case, enumerating matches of patterns of size p can require a number of candidate assignments exponential in p. The memoization map stores $O(|\Sigma|(p+2))$ status entries; retaining the complete candidate pool additionally requires space proportional to the stored candidate representations. The K selected graphs require space proportional to their actual stored edge sets.

Analysis of HeuIChase. We present the detailed analysis below.

Correctness. We analyze HeuIChase under the restricted tree/arborescence semantics adopted in Section VI: each returned witness G_s is a verified tree witness rooted at v_t, and each returned grounded graph G_g is a weighted arborescence of $G^L(v_t)$ rooted at v_t and containing G_s. Grounding in HeuIChase is realized by supporting-arborescence expansion using observed edges.

Lemma 8: *Every pair (G_s, G_g) returned by* HeuIChase *is a feasible grounded witness pair in the tree/arborescence setting.* □

Proof. Let (G_s, G_g) be a pair returned by HeuIChase .

(1) **Witness Correctness.** L-Chase generates witnesses by matching a tree pattern P against sampled trees from $G^L(v_t)$, and retains a candidate only if it passes VerifyWitness. Hence every returned G_s is a factual or counterfactual witness for $Q(M, v_t, G)$. By construction of HeuIChase, G_s is a rooted tree containing v_t.

(2) **Containment and Arborescence Structure.** Given a verified witness G_s, HeuIChase contracts G_s into a supernode s and applies Edmonds' algorithm on the contracted graph. Edmonds' algorithm returns a maximum-weight arborescence rooted at s over the contracted graph. Expanding the supernode s back to the original witness edges yields a connected rooted arborescence G_g in the original graph domain. Since the contraction preserves the internal structure of G_s, we have $G_s \subseteq G_g$. Moreover, since G_s is rooted at v_t, the expanded arborescence G_g is rooted at v_t in the original graph.

(3) **Grounding Correctness.** Let $\Sigma'(G_g) = \{\varphi \in \Sigma \mid G_g \models \varphi\}$ be the set of constraints grounded by G_g. After constructing G_g, HeuIChase evaluates which constraints in Σ are satisfied by the resulting arborescence and retains the pair only when $\Sigma'(G_g) \neq \emptyset$.

For every $\varphi = (P, c) \in \Sigma'(G_g)$, the graph G_g contains a co-occurring match of P and c under the same variable assignment; hence G_g grounds G_s under $\Sigma'(G_g)$.

Therefore, G_s is a verified witness, G_g is a rooted arborescence containing G_s, and G_g grounds G_s under a nonempty subset of constraints. Thus (G_s, G_g) is a feasible grounded witness pair in the tree/arborescence setting. □

Remark. Since HeuIChase uses only observed edges, $\Delta E = \emptyset$, and each retained pair is 0-grounded by the constraints satisfied in G_g. It heuristically realizes L-Backchase by expanding a verified tree witness into a supporting weighted arborescence.

Approximability. HeuIChase adopts the same UpdateWK routine as ApxIChase. Let $\mathcal{C}$ denote the candidate pool of grounded witness pairs generated by HeuIChase and available at request time.

Proposition 9: *Let $\mathcal{P}^* \in \arg\max_{\mathcal{P}' \subseteq \mathcal{C},\, |\mathcal{P}'| \leq K} F(\mathcal{P}')$ be an optimal size-K witness-pair set over the current candidate pool $\mathcal{C}$. Then* UpdateWK *returns a set W_K such that $F(W_K) \geq (1-1/e) \cdot F(\mathcal{P}^*)$.* □

Proof. For each candidate pair $(G_s, G_g) \in \mathcal{C}$, the score $q(G_s, G_g)$ is fixed once the pair is generated; for HeuIChase, $\beta = 0$ and the score is determined by witness conciseness. Hence the term $\sum_{(G_s,G_g)\in\mathcal{P}} q(G_s, G_g)$ is modular over the candidate set.

Moreover, $\mathsf{cov}(\mathcal{P}) = \frac{\left|\bigcup_{(G_s,G_g)\in\mathcal{P}} \Sigma'(G_g)\right|}{|\Sigma|}$ is a normalized set-coverage function, and is therefore monotone submodular. Thus the overall set objective $F(\mathcal{P}) = \sum_{(G_s,G_g)\in\mathcal{P}} q(G_s, G_g) + \gamma \cdot \mathsf{cov}(\mathcal{P})$ is monotone submodular over $\mathcal{C}$.

Under the cardinality constraint $|\mathcal{P}| \leq K$, the classical greedy result of Nemhauser et al. implies that greedy maximization achieves a $(1-1/e)$-approximation to the optimum over $\mathcal{C}$. Since UpdateWK greedily refreshes the top-K set by repeatedly selecting the pair with the largest marginal gain under $F(\mathcal{P})$, the bound follows. □

Remark. This approximation guarantee is conditional on the candidate pool $\mathcal{C}$ generated by HeuIChase. Since tree sampling and weighted arborescence construction are heuristic, the guarantee does not imply a constant-factor approximation to the global optimum over all feasible grounded witness pairs.

Cost Analysis. We separate the costs of tree sampling, candidate matching, arborescence construction, validation, and top-K selection. Let m be the number of sampled trees per round, L the number of rounds, p the maximum pattern size, $|\Sigma|$ the number of constraints, and $|\mathcal{C}|$ the number of retained candidate pairs. The implementations of tree sampling, pattern matching, and optimum arborescence computation determine their respective operation counts; the current algorithm description does not prove a uniform $O(p^2)$ bound per tree-pattern match or a uniform $O(|G^L(v_t)| \log |G^L(v_t)|)$ bound per arborescence on every input. We therefore refrain from combining those unverified component bounds into an end-to-end asymptotic guarantee. The final greedy selection over the pool costs $O(K|\mathcal{C}||\Sigma|)$ with direct coverage-set scans, as in Theorem 3. The algorithm applies one arborescence computation to each retained witness and checks the selected constraints on the resulting graph. This component-wise description isolates the sources of runtime shown in the experiments. Its primary computational advantage comes from restricting candidate exploration to sampled trees and using observed-edge arborescence expansion.

The memory cost depends on how candidates are materialized.

Materialized Implementation. If sampled trees and candidate pairs are stored explicitly, the memory cost per request is $O\big(|G^L(v_t)| + m|G^L(v_t)| + |\mathcal{C}|\,|G^L(v_t)| + K|G^L(v_t)|\big)$. The four terms correspond to storing the L-hop neighborhood, the sampled trees, the current candidate pool, and the selected top-K witness pairs, respectively.

Compressed Implementation. If candidate pairs are stored only through parent pointers, scores, and grounded-constraint identifiers, the bookkeeping term reduces to $O(|\mathcal{C}|)$ on top of shared graph storage, yielding $O\big(|G^L(v_t)| + m|G^L(v_t)| + |\mathcal{C}| + K|G^L(v_t)|\big)$. If sampled trees are generated and processed on-the-fly, the term $m|G^L(v_t)|$ further reduces to $O(|G^L(v_t)|)$ working memory. Unlike ApxIChase, no additional $O(|\Sigma|k)$ space is needed, since HeuIChase does not explicitly maintain grounded-edge insertion sets or budget-tracking structures for individual candidates.

Dataset	#rules (before → after)	Avg $\lvert P\rvert$ size	Active ratio
DBLP	64 → 20	3.4	0.54
Cora	25 → 20	3	0.46
MUTAG	9 → 9	3.56	0.69
BAShape	5 → 5	2.8	0.60
ATLAS	28 → 12	3.75	0.61
ogbn-Papers100M	52 → 20	3.35	0.23
Tree-Cycles	18 → 14	3.14	0.58

TABLE IV
CONSTRAINT STATISTICS ACROSS DATASETS.

Algorithm 6 : paraIChase: Parallel Witness Generation

Input: G, constraint set Σ, M, target node set V_T, L,K,k, #workers w, ALG $\in \{$APXC, HEUC, EXH, GEX, PGX$\}$
Output: $\{(v_t, W_K(v_t)) \mid v_t \in V_T\}$

```
Coordinator computes G^L(V_T) = {(v_t, G^L(v_t)) | v_t ∈ V_T}
Π ← LOADBALANCE(G^L(V_T), w) ▷ load balance by |G^L(v_t)|
for i = 1 to w do in parallel
    W_K^(i) ← ∅
    for all (v_t, G^L(v_t)) ∈ Π_i do
        Execute ALG on (v_t, G^L(v_t)) to obtain W_K(v_t)
        W_K^(i) ← W_K^(i) ∪ {(v_t, W_K(v_t))}
return ⋃_{i=1}^{w} W_K^(i)
```

Fig. 13. Algorithm paraIChase: Parallelized witness generation framework with load-balanced subgraph distribution.

C. Appendix C: Complementary Experimental Study and Additional Discussion

Discussion: Applications of Grounded Witnesses. We briefly discuss why k-grounded witnesses are useful in practice. (1) High interpretability. It readily suggests critical fraction G_s of the original graph G that are responsible to $Q(M, v_t, G)$, and an extended counterpart G_g to ensure a co-occurrence of matches of P and c, hence directly grounding G_s to user suggested domain evidence as specified by φ. The constraint $\varphi \in \Sigma$, in turn, contributes to rule-based GNN interpretation as logical rules. (2) Data preparing and refinement for graph learning. While conventional graph enrichment focuses on maximizing data completeness rather than "ML-aware", the additional edges ΔE readily suggest "how" G should be modified to include missing data, that are specifically relevant to the decision making process of M. This also indicates useful training and testing triples for GNNs. (3) Adversarial graph learning. The ΔE and G_s together indicate "vulnerable" area that may be attacked or poisoned to affect GNN M's output, hence robust training can be localized to avoid expensive defense cost.

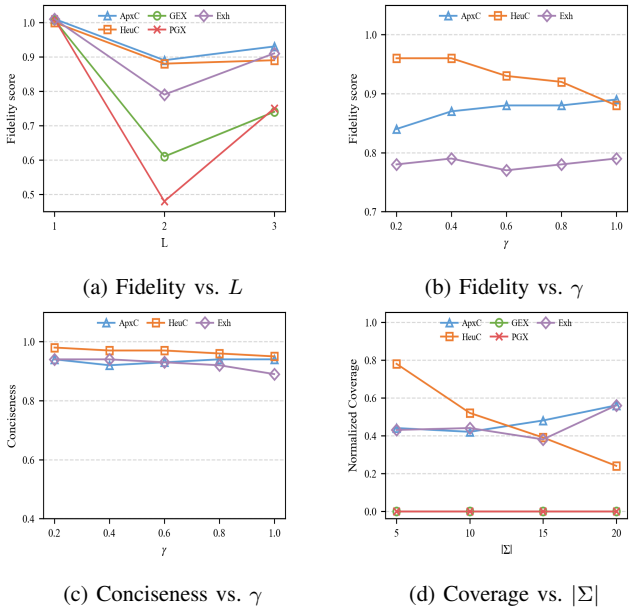
Constraint Statistics. Table IV summarizes the basic statistics of the constraint pools used in our experiments across datasets. Specifically, "before $\rightarrow$ after" reports the number of constraints before and after pruning, "Avg $|P|$ size" denotes the average number of antecedent edges in the final rule pool, and "Active ratio" is the average fraction of active constraints per workload over the final rule pool. These statistics show that the final constraint pools remain compact after pruning while still retaining a nontrivial fraction of active constraints for grounding. They also indicate that the antecedent patterns used in practice are local and moderate in size, which is consistent with our focus on bounded witness grounding over local graph neighborhoods.

Parallel Algorithm. Fig. 13 presents paraIChase, a parallel witness generation framework. A coordinator process induces L-hop subgraphs $\{(v_t, G^L(v_t))|v_t \in V_T\}$ and partitions them across w workers via First-Fit Decreasing bin-packing based on subgraph size $|G^L(v_t)|$, using a min-heap over worker loads to mitigate stragglers. Each worker independently runs one of the base algorithms ALG $\in$ {ApxC, HeuC, Exh, GEX, PGX} on its assigned subgraphs. Global parameters are shared; workers access subgraphs via indexed slicing to avoid duplication. Verification is performed in mini-batches of 32 per worker to amortize inference costs. Coordinator aggregates results $W_K(v_t)$ to compute final metrics.

Exp-4: Complementary Experiments on DBLP. We conduct several complementary experiments on the *DBLP* dataset to further examine the sensitivity of our methods to model depth, objective weighting, and constraint pool size.

Fidelity vs. L. We vary the model depth $L \in \{1, 2, 3\}$ while keeping all other factors fixed. As shown in Fig. 14(a), our methods maintain relatively high fidelity as L increases, although all methods exhibit some drop from L=1 to larger neighborhoods. In particular, ApxIChase and HeuIChase remain consistently more faithful than GNNExplainer and PGExplainer at L=2 and L=3. This suggests that incorporating grounding into witness generation does not prevent the selected witnesses from preserving the model's prediction behavior, even when the receptive field becomes larger.

Fidelity vs. γ. We vary $\gamma \in \{0.2, 0.4, 0.6, 0.8, 1\}$ while maintaining $\alpha = \beta = \frac{1}{2}$. As shown in Fig. 14(b), the fidelity of ApxIChase increases slightly as γ grows, whereas HeuIChase shows a mild decline. Meanwhile, naiveChase remains largely stable across different values of γ. Overall, these results indicate that increasing the weight of coverage in the set objective does not cause a drastic or systematic loss of fidelity, although different witness-generation strategies still respond differently to this trade-off.



(a) Fidelity vs. L (b) Fidelity vs. γ

(c) Conciseness vs. γ (d) Coverage vs. $|\Sigma|$

Fig. 14. Complementary Experiments

Conciseness vs. γ. Fig. 14(c) shows that conciseness remains high and largely stable as γ varies. Both ApxIChase and HeuIChase maintain consistently high scores across all settings. In particular, HeuIChase remains slightly higher, indicating that although its arborescence expansion may produce a larger grounded graph, the selected witness itself is still compact. By contrast, naiveChase shows a mild decline as γ increases, suggesting that placing more emphasis on coverage may introduce slightly larger witnesses. We report only our methods here, since γ is specific to our objective.

Coverage vs. $|\Sigma|$. We further vary the size of the constraint pool, with $|\Sigma| \in \{5, 10, 15, 20\}$, and report normalized coverage in Fig. 14(d). Overall, the baselines remain at zero, while our methods achieve substantially higher coverage. For ApxIChase and naiveChase, the trend is not strictly monotone: enlarging the constraint pool changes both the number of active constraints and the subset that can be covered, so the normalized coverage may increase or decrease depending on the specific pool. In contrast, HeuIChase decreases steadily as $|\Sigma|$ grows, indicating that its tree-biased candidates cover a shrinking fraction of the enlarged constraint pool. This result also highlights why we report normalized coverage rather than raw counts: the full constraint pool may contain many constraints that are irrelevant to the current workload.